



DB MID 2

Database Design:

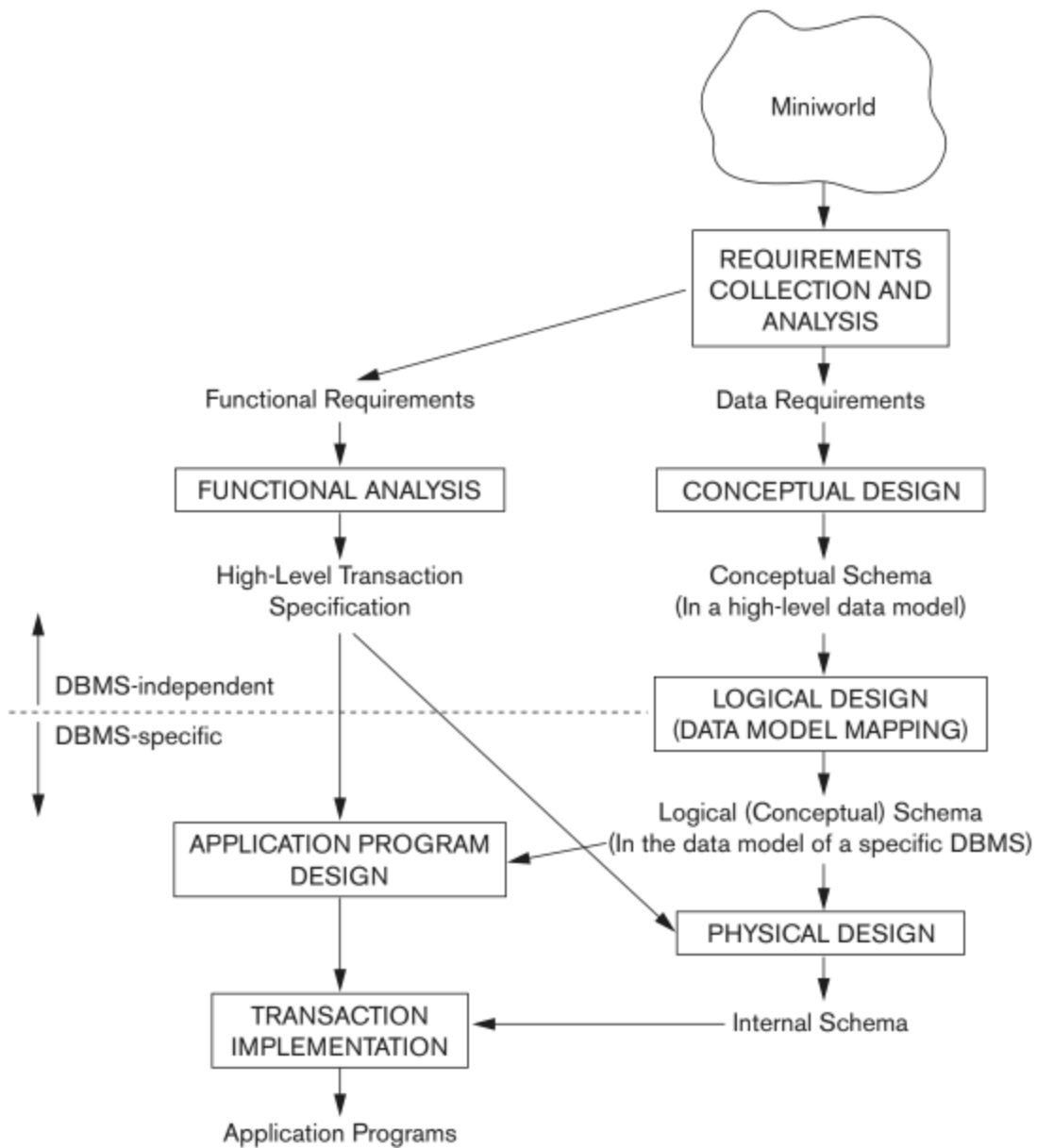


Figure 3.1
A simplified diagram to illustrate the main phases of database design.

Phase	Focus	Output	Nature
Requirements Collection & Analysis	Understand data & functional needs	User Requirements	DBMS-independent
Conceptual Design	Model data logically	Conceptual Schema	DBMS-independent

Phase	Focus	Output	Nature
Logical Design	Map to DBMS data model	Logical Schema	DBMS-specific
Physical Design	Optimize storage & access	Internal Schema	DBMS-specific
Application Design	Implement transactions	Application Programs	DBMS-specific

Concept of ERD (Entity–Relationship Diagram)

1. What is an ERD?

An **Entity–Relationship Diagram (ERD)** is a **visual representation of data** and the relationships between data entities in a database system.

It helps **design the structure of a database** before actually creating it.

In simple words:

ERD shows **what data** will be stored (entities and attributes) and **how** those data items are **related** (relationships).

2. Main Components of an ERD

An ERD is made up of **three basic components**:

Component	Description	Example
Entity	A real-world object or concept that you want to store information about.	Student, Course, Teacher, Department
Attribute	The data we store about an entity.	Name, Roll_no, Age, Address
Relationship	How entities are related to one another.	Student enrolls in Course

3. Entities

- **Definition:** An **entity** represents an object or concept about which you want to store data.
- **Types of Entities:**
 - **Strong Entity** → Exists independently.

Example: **Student**, **Course**

- **Weak Entity** → Depends on another entity for identification.

Example: **Dependent** depends on **Employee** (for ID).

Representation: Rectangle

→ Example:  **STUDENT**

4. Attributes

- **Definition:** Characteristics or properties that describe an entity.

Types of Attributes:

Type	Description	Example
Simple / Atomic	Cannot be divided further	Age, Name
Composite	Can be divided into subparts	Address → Street, City, Zip
Derived	Computed from another attribute	Age (from Birth_date)
Multivalued	May have multiple values	Phone_numbers, Degrees
Key Attribute	Unique identifier	Roll_no, Employee_ID

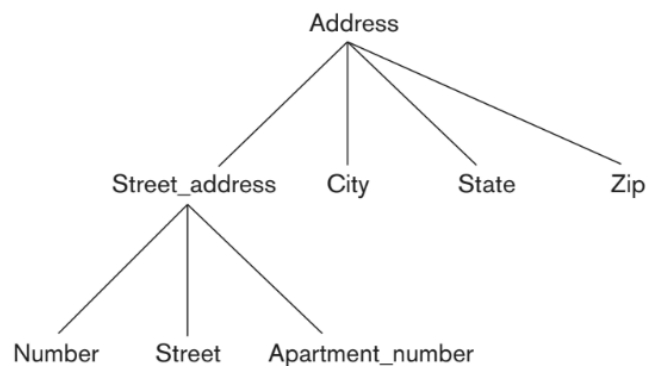


Figure 3.4
A hierarchy of
composite attributes.

Representation: Oval connected to the entity.

5. Relationships

- **Definition:** Associations among entities.
 - Example:

STUDENT enrolls in **COURSE**

Types of Relationships:

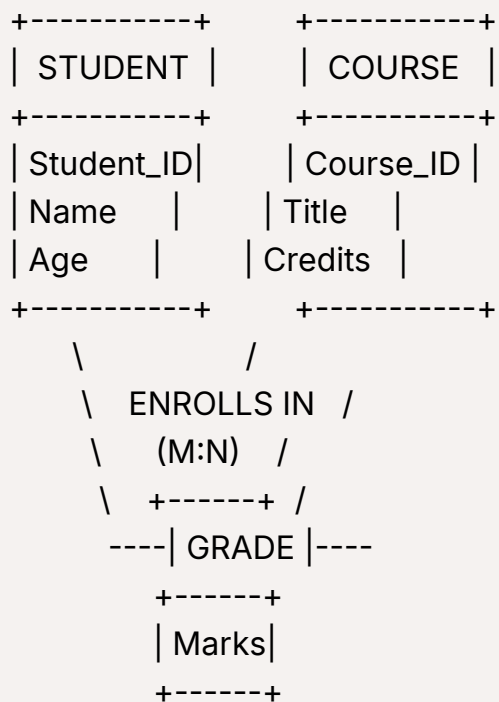
Type	Description	Example
One-to-One (1:1)	One entity instance relates to one other	Person ↔ Passport
One-to-Many (1:N)	One entity relates to many others	Teacher → Students
Many-to-Many (M:N)	Many entities relate to many others	Students ↔ Courses

Representation: Diamond shape between entities.

6. Keys

- **Primary Key** → Unique identifier of an entity (**Student_ID**).
- **Foreign Key** → Attribute that creates a link between two entities (**Course_ID** in Enrollment table).
- **Candidate Key** → A possible choice for primary key.

7. Example of an ERD



Explanation:

- Each **student** can enroll in **many courses**.
- Each **course** can have **many students**.
- This M:N relationship is represented by a separate **relationship entity** called **GRADE**.

8. Purpose of an ERD

- ✓ To **understand data requirements** before building the database.
- ✓ To **avoid redundancy** and design normalized data.
- ✓ To serve as a **blueprint** for database implementation.
- ✓ To **communicate clearly** between developers, analysts, and stakeholders.

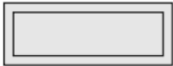
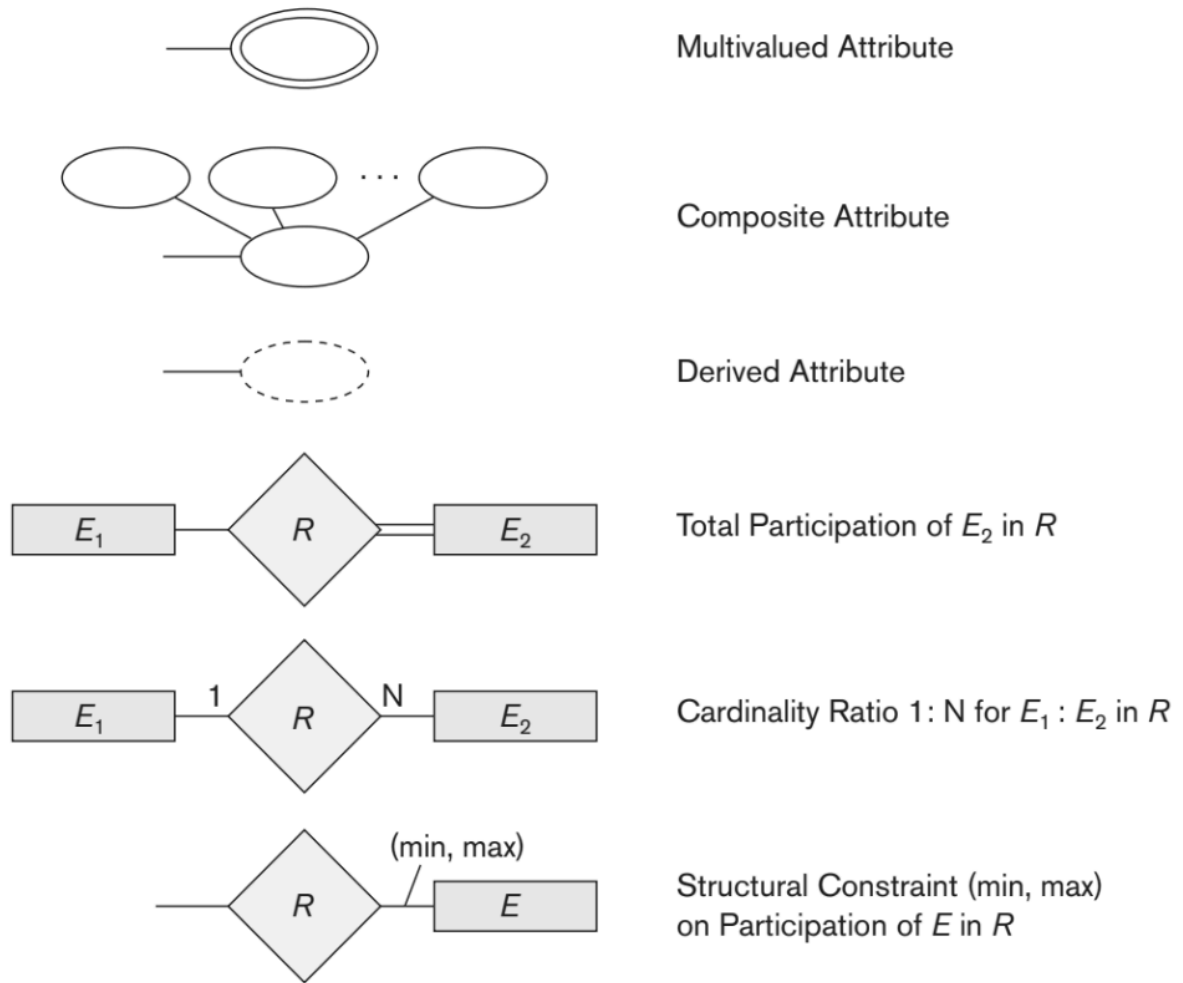
Symbol	Meaning
	Entity
	Weak Entity
	Relationship
	Identifying Relationship
	Attribute
	Key Attribute

Figure 3.14

Summary of the notation for ER diagrams.



Types of Relationships in ERD

There are **three main types** of relationships:

Relationship Type	Example	Symbolic Form
One-to-One	1:1	Each entity instance in A is related to at most one in B
One-to-Many	1:N	Each entity in A can relate to many in B, but each B relates to only one A
Many-to-Many	M:N	Many in A can relate to many in B

1. One-to-One (1:1) Relationship

Example:

Each **Person** has **one Passport**, and each **Passport** belongs to **one Person**.

Entities and Attributes

Entity	Attributes
Person	Person_ID (PK), Name, Age
Passport	Passport_ID (PK), Issue_Date, Expiry_Date, Person_ID (FK)

```
+-----+ +-----+
| PERSON |1----1| PASSPORT |
+-----+ +-----+
| Person_ID |←---→| Passport_ID |
| Name      |      | Issue_Date |
| Age       |      | Expiry_Date |
+-----+ +-----+
```

Conversion to Tables

Usually, you can **combine (reduce)** the two tables into one **if**:

- Both have a **strict 1:1** relationship.
- Every person has a passport and vice versa.

Can be reduced?

Yes, merge into one table like:

```
| Person_ID | Name | Age | Passport_ID | Issue_Date | Expiry_Date |
```

But if **some people don't have passports**, you should **keep them separate**.

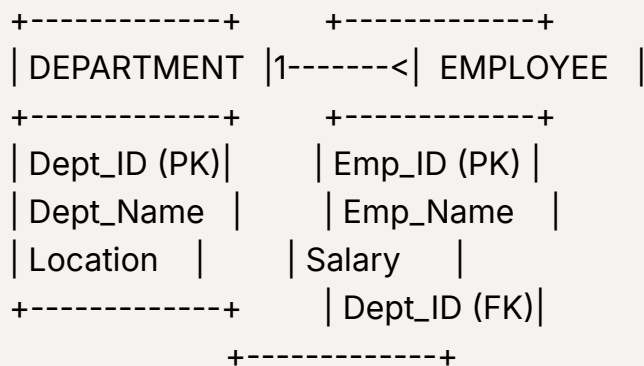
2. One-to-Many (1:N) Relationship

Example:

One **Department** has **many Employees**, but each **Employee** belongs to **only one Department**.

Entities and Attributes

Entity	Attributes
Department	Dept_ID (PK), Dept_Name, Location
Employee	Emp_ID (PK), Emp_Name, Salary, Dept_ID (FK)



Conversion to Tables

- Create **two tables**.
- The **foreign key (Dept_ID)** is added to the **many-side (Employee)** table.

👉 Can be reduced?

❌ **No**, usually **cannot reduce** because:

- One department has multiple employees.
- If merged, data redundancy will occur (same Dept info repeated many times).

3. Many-to-Many (M:N) Relationship

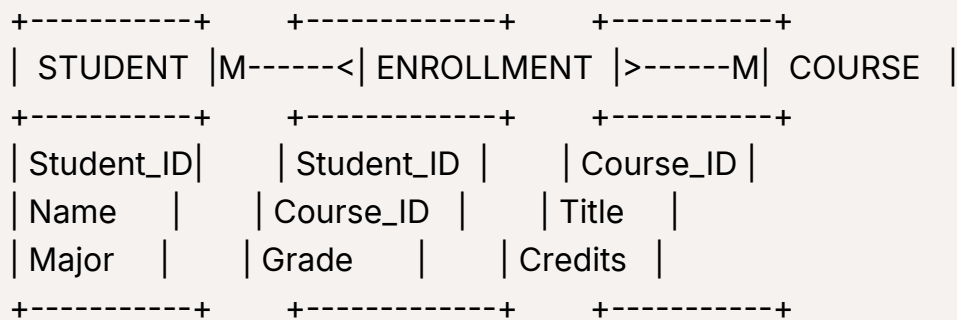
📖 Example:

Students **enroll** in many **Courses**, and each **Course** has many **Students**.

Entities and Attributes

Entity	Attributes
Student	Student_ID (PK), Name, Major
Course	Course_ID (PK), Title, Credits
Enrollment (<i>relationship entity</i>)	Student_ID (FK), Course_ID (FK), Grade

ER Diagram



Conversion to Tables

- Create **three tables**:
 1. **Student**
 2. **Course**
 3. **Enrollment** (a separate table for M:N relationship)

👉 Can be reduced?

✗ No, cannot reduce, because:

- A student can take multiple courses.
- A course can have multiple students.
- Relationship data (like **Grade**) is specific to the pair (Student–Course).

Relationship Type	Example	ERD Representation	Can Tables Be Reduced?	Reason
1:1	Person ↔ Passport	One-to-One	✅ Sometimes	If both entities always exist together
1:N	Department ↔ Employee	One-to-Many	❌ No	Repetition of data if merged
M:N	Student ↔ Course	Many-to-Many	❌ No	Need separate linking (relationship) table

Weak Entity Types

Definition

- A **weak entity type** is an entity type that **does not have a key attribute of its own**.
- **Strong (regular) entity types** have their own key attributes.

Identification

- A weak entity is identified by:
 1. Being **related to an owner (identifying) entity type**
 2. Using a **partial key** (an attribute that is unique only within the context of a specific owner entity)

Key Terms:

Term	Meaning
Owner / Identifying Entity	Strong entity type that identifies the weak entity
Identifying Relationship	Relationship linking weak entity to owner
Partial Key	Attribute(s) uniquely identifying weak entity among entities of the same owner
Existence Dependency	Weak entity cannot exist without an owner (total participation)

Weak (Identifying) Relationship

- The **relationship type** that **links a weak entity** to its **owner (strong) entity**.
- Its main purpose is to **identify the weak entity** because the weak entity **cannot exist independently**.
- **Properties:**
 - Total participation on the **weak entity side** (min = 1)
 - Owner entity side may have optional participation (min = 0)
 - Usually represented as a **double diamond** in ER diagrams
- **Example:**
 - **DEPENDENT** (weak entity)
 - **EMPLOYEE** (owner entity)
 - Relationship: **HAS_DEPENDENT** → **identifying (weak) relationship**

Weak Link

- Informally, the **weak link** refers to a **specific connection (instance) in the identifying relationship** that allows a particular weak entity to be identified.
- It is the **actual association** between a **weak entity instance** and its **owner entity instance**.
- In other words:
 - **Weak relationship = type-level description**
 - **Weak link = instance-level connection**

Example:

- **Employee e1** has two dependents: **John** and **Mary**
- Each weak entity instance (**John** , **Mary**) is connected to **e1** through a **weak link** (specific relationship instance)
- The **collection of all weak links** forms the **weak relationship set**

Example: DEPENDENT

- **Owner entity:** EMPLOYEE
- **Weak entity:** DEPENDENT
- **Attributes of DEPENDENT:** Name, Birth_date, Sex, Relationship
- **Partial key:** Name (assuming no two dependents of the same employee share a name)
- **Relationship:** HAS_DEPENDENT (identifying relationship)

Observation:

Two dependents of different employees may have identical attributes; they are only distinguishable **in combination with their owner**.

ER Diagram Representation

- **Weak entity:** Double rectangle
- **Identifying relationship:** Double diamond
- **Partial key:** Dashed underline
- **Total participation:** Double line connecting weak entity to identifying relationship

Alternative Representation

- Weak entities can sometimes be modeled as **complex or multivalued attributes** of the owner entity.
- Example: Dependents as a multivalued composite attribute of EMPLOYEE with components: Name, Birth_date, Sex, Relationship.

Design Choice:

- Use a weak entity type if it **participates independently** in relationships other than its identifying relationship.

Additional Notes

- Weak entities can exist in **multiple levels** (an owner may itself be weak).
- A weak entity may have **multiple identifying entity types**.
- Identifying relationship can be of **degree higher than two**.

ERD Example: Instructor Advises Student

1. Entities and Attributes

We have **two entities** here:

INSTRUCTOR and **STUDENT**.

Entity 1: INSTRUCTOR

Attribute	Description	Key Type
Instructor_ID	Unique identifier for each instructor	Primary Key (PK)
Name	Instructor's name	—
Salary	Instructor's salary	—

Symbol in ERD: Rectangle labeled *INSTRUCTOR*

Attributes are ovals attached to it, with **Instructor_ID** underlined (as the primary key).

Entity 2: STUDENT

Attribute	Description	Key Type
Student_ID	Unique identifier for each student	Primary Key (PK)
Name	Student's name	—
Credits	Number of credits the student has earned	—

Symbol in ERD: Rectangle labeled *STUDENT*

Attributes are ovals attached, with **Student_ID** underlined.

2. Relationship: ADVISES

- Relationship **name:** **ADVISES**
- **Meaning:**
 - One **Instructor** can advise **many Students**.
 - Each **Student** is advised by **only one Instructor**.

Hence, the **cardinality (mapping)** is:

| Instructor (1) — (N) Student

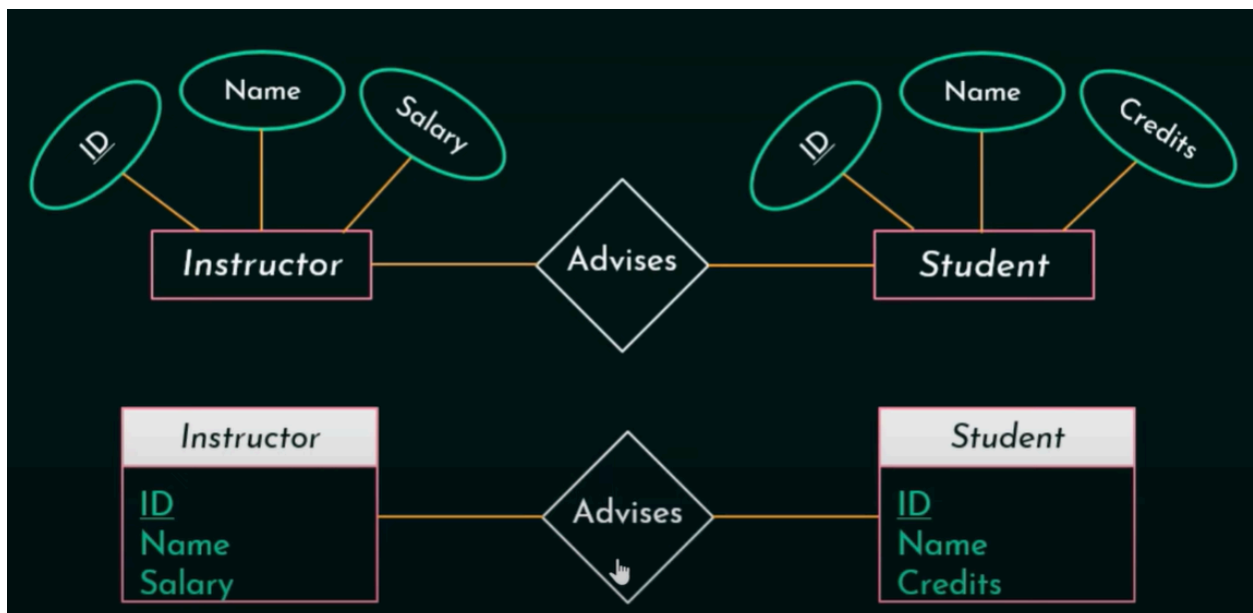
This makes it a **one-to-many (1:N)** relationship.

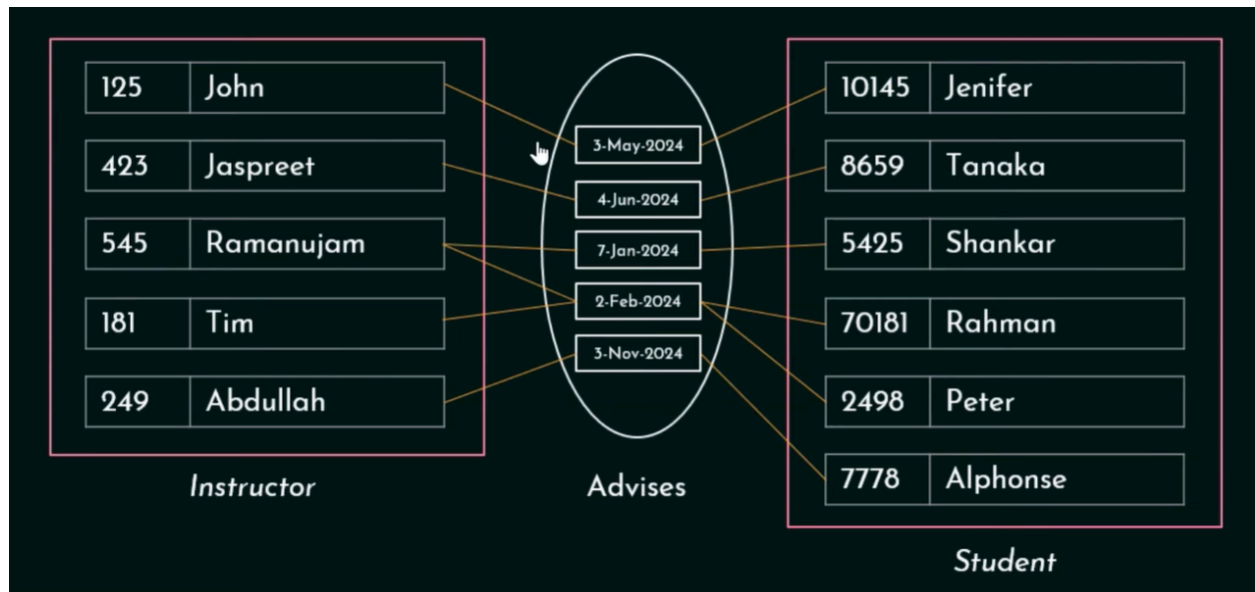
3. Descriptive Attribute

The relationship **ADVISES** has a **descriptive attribute** → **Date**.

- Meaning: It tells **when** the instructor started advising the student (or when the advice was given).
- Since the attribute **Date** **belongs to the relationship** (not to either entity), it is drawn as an **oval connected to the diamond** labeled **ADVISES**.

Descriptive attributes describe **properties of a relationship** itself, not of any single entity.





Example:

Overview

This **ER Diagram** models a **company database schema** that keeps track of:

- Employees
- Departments
- Projects
- Dependents
- Relationships between them (like supervision, management, and project assignment)

It uses the **(min, max)** notation to define **cardinality constraints** — that is, how many instances of one entity can be related to instances of another.

ENTITIES & ATTRIBUTES

1. EMPLOYEE

Attributes:

- **Fname** → First name
- **Minit** → Middle initial
- **Lname** → Last name
- **Ssn** → Social security number (Primary Key)
- **Bdate** → Birth date
- **Address** → Employee's address
- **Sex** → Gender
- **Salary** → Employee's salary

Each employee has a unique **Ssn**.

2. DEPARTMENT

Attributes:

- **Name** → Department name (unique)
- **Number** → Department number (Primary Key)
- **Locations** → Multi-valued attribute (a department may be in multiple locations)
- **Number_of_employees** → Derived attribute (shown dotted; derived from counting employees who work in that department)

3. PROJECT

Attributes:

- **Name** → Project name (unique)
- **Number** → Project number (Primary Key)
- **Location** → Location where the project is carried out

4. DEPENDENT

Attributes:

- **Name** → Dependent's name
- **Sex** → Gender

- **Birth_date** → Date of birth
- **Relationship** → Relationship to the employee (e.g., child, spouse)

Each dependent is uniquely identified **in the context of an employee** (i.e., **Employee.Ssn + Dependent.Name** forms a unique pair).

RELATIONSHIPS

1. WORKS_FOR

- **Between:** **EMPLOYEE** and **DEPARTMENT**
- **Meaning:** Each employee works for exactly one department.
- **Attributes:** **Start_date** — when the employee started working for that department.
- **Cardinalities:**
 - **(1,1)** on **EMPLOYEE** side → every employee must work for exactly one department.
 - **(4,N)** on **DEPARTMENT** side → each department must have at least 4 employees, and can have many.

2. MANAGES

- **Between:** **EMPLOYEE** and **DEPARTMENT**
- **Meaning:** One employee manages one department.
- **Attributes:** **Start_date** — when the management assignment began.
- **Cardinalities:**
 - **(0,1)** on **EMPLOYEE** side → an employee **may** manage **at most one** department, or may manage none.
 - **(1,1)** on **DEPARTMENT** side → each department **must have** exactly one manager.

3. SUPERVISION

- **Between:** **EMPLOYEE** (Supervisor) and **EMPLOYEE** (Supervisee)

- **Meaning:** Models who supervises whom.
- **Cardinalities:**
 - (0,N) on Supervisor side → an employee may supervise multiple employees, or none.
 - (0,1) on Supervisee side → an employee may be supervised by at most one supervisor.

This is a **recursive (self-referential) relationship** on the **EMPLOYEE** entity.

4. WORKS_ON

- **Between:** **EMPLOYEE** and **PROJECT**
- **Meaning:** Employees can work on multiple projects.
- **Attributes:** **Hours** — number of hours per week spent on the project.
- **Cardinalities:**
 - (0,N) on **EMPLOYEE** side → an employee may work on several projects, or none.
 - (1,N) on **PROJECT** side → each project must have at least one employee and can have many.

This is a **many-to-many** relationship.

5. CONTROLS

- **Between:** **DEPARTMENT** and **PROJECT**
- **Meaning:** Each department controls one or more projects.
- **Cardinalities:**
 - (1,1) on **PROJECT** side → each project is controlled by exactly one department.
 - (0,N) on **DEPARTMENT** side → a department may control many projects or none.

6. DEPENDENTS_OF

- **Between:** `EMPLOYEE` and `DEPENDENT`
- **Meaning:** Each employee can have dependents.
- **Cardinalities:**
 - `(0,N)` on `EMPLOYEE` side → an employee may have no dependents or many.
 - `(1,1)` on `DEPENDENT` side → each dependent must belong to exactly one employee.

OTHER DETAILS

Derived Attribute

- `Number_of_employees` in `DEPARTMENT` — shown with a **dotted ellipse**, indicating that it can be computed from the database (e.g., count of employees working in that department).

Multi-Valued Attribute

- `Locations` of `DEPARTMENT` — double oval, showing that a department can exist in multiple places.

Role Names

Used in relationships to distinguish roles when entities of the same type are related or to clarify relationship meaning:

- In **WORKS_FOR** → Employee, Department
- In **SUPERVISION** → Supervisor, Supervisee
- In **WORKS_ON** → Worker, Project
- In **MANAGES** → Manager, Managed Department

Conceptual Summary:

Entity	Relationship	Meaning	Attributes in Relationship	Cardinality
EMPLOYEE	WORKS_FOR	Employee works for a Department	Start_date	(1,1)–(4,N)
EMPLOYEE	MANAGES	Employee manages Department	Start_date	(0,1)–(1,1)
EMPLOYEE	SUPERVISION	Employee supervises Employee	—	(0,N)–(0,1)
EMPLOYEE	WORKS_ON	Employee works on Project	Hours	(0,N)–(1,N)
DEPARTMENT	CONTROLS	Department controls Project	—	(0,N)–(1,1)
EMPLOYEE	DEPENDENTS_OF	Employee has Dependents	—	(0,N)–(1,1)

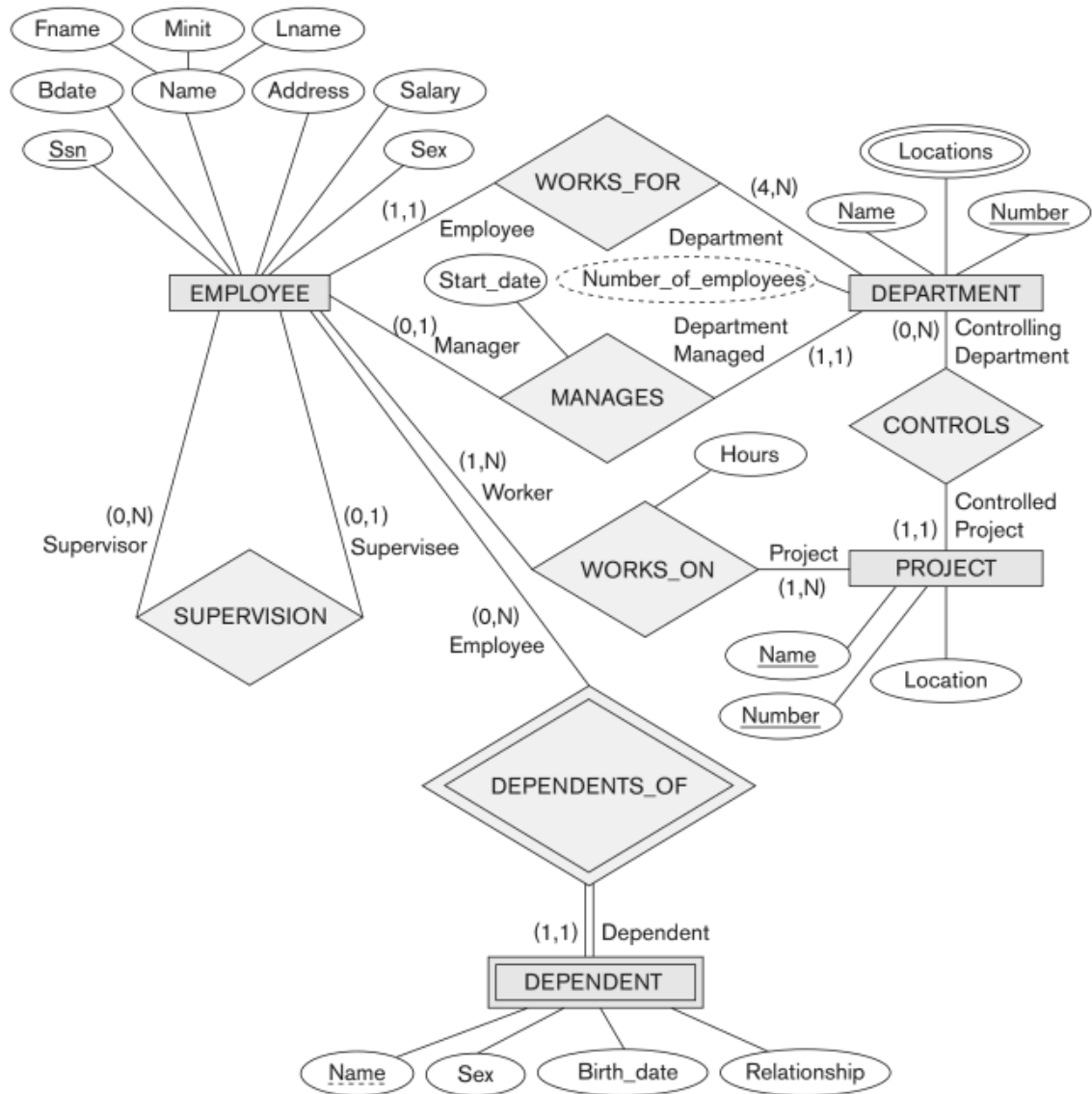
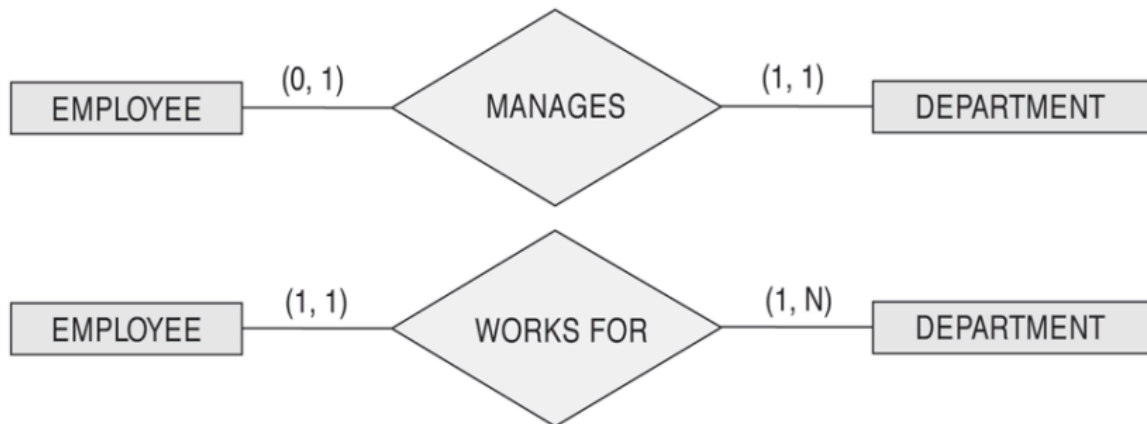


Figure 3.15

ER diagrams for the company schema, with structural constraints specified using (min, max) notation and role names.

The (min,max) notation for relationship constraints



Read the min,max numbers next to the entity type and looking **away from** the entity type

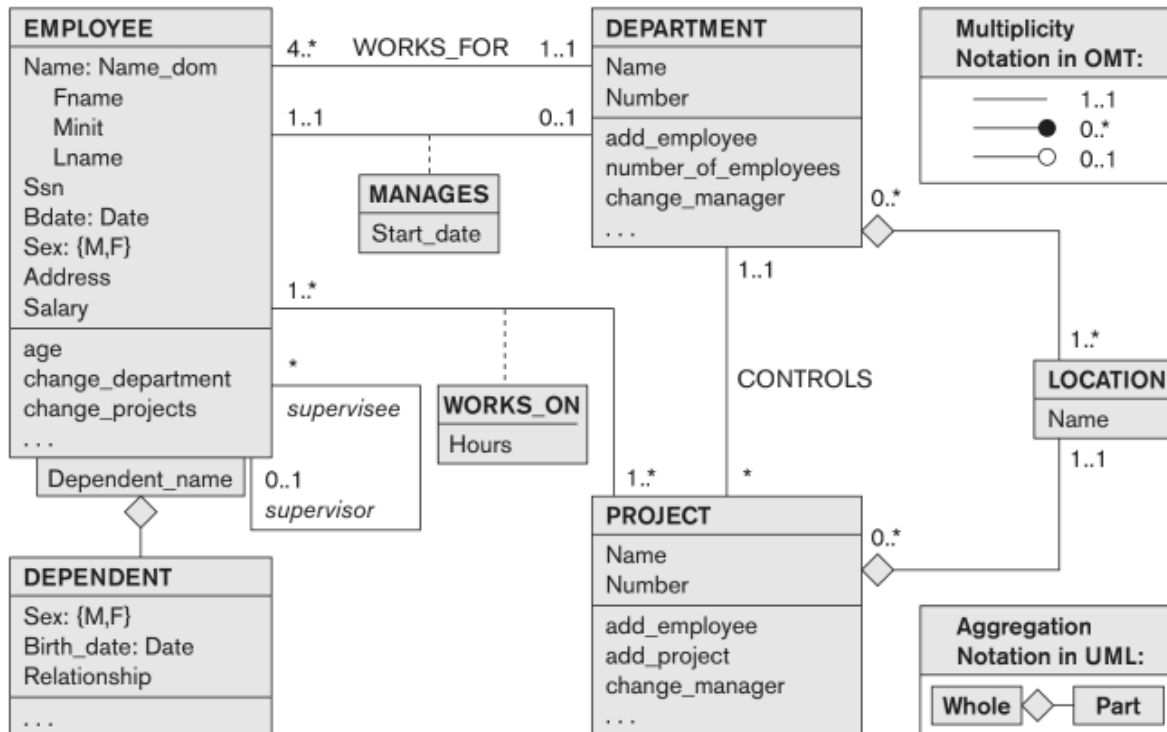


Figure 3.16
The COMPANY conceptual schema in UML class diagram notation.

Binary, Ternary, and N-ary Relationships:

Overview

This diagram explores **how to represent relationships among three entities**:

- **SUPPLIER** — supplies something
- **PART** — the item being supplied
- **PROJECT** — the project receiving the part

The example is about modeling **who supplies which part to which project** and in what **quantity**.

The three subfigures (a), (b), and (c) show **different ways** to represent this same scenario.

Figure 3.17(a): A Ternary Relationship

Entities:

1. SUPPLIER

- Attribute: **Sname** (Supplier name)

2. PART

- Attribute: **Part_no** (Part number)

3. PROJECT

- Attribute: **Proj_name** (Project name)

Relationship: **SUPPLY**

- **Type: Ternary relationship** (involves three entities simultaneously)
 - **Attributes:** **Quantity** — number of parts supplied
-

Meaning

The ternary relationship means:

| A supplier supplies a certain part to a particular project in a specific quantity.

Formally:

SUPPLY(SUPPLIER, PART, PROJECT, Quantity)

This tells us that the **relationship involves all three entities together** — not just pairs of them.

Example:

Supplier	Part	Project	Quantity
S1	P2	J3	500
S1	P1	J1	200

Meaning:

- Supplier **S1** supplies **500 units of Part P2** to **Project J3**.

- Supplier **S1** supplies **200 units of Part P1** to **Project J1**.

If you remove any of the three — the meaning changes.

So this cannot be represented accurately by separate binary relationships.

Key Point:

- **Ternary relationships** express dependencies among **three entities at once**.
- You **cannot** simply replace them by three binary relationships (Supplier–Part, Supplier–Project, Part–Project) because that would lose important dependency information.

Figure 3.17(b): Three Binary Relationships (NOT Equivalent)

Here we have **three separate binary relationships** connecting the same entities.

Entities: Same as before

- SUPPLIER (**Sname**)
- PART (**Part_no**)
- PROJECT (**Proj_name**)

Relationships:

1. **CAN_SUPPLY** — between SUPPLIER and PART
(which parts each supplier can supply)
2. **USES** — between PROJECT and PART
(which parts each project needs)
3. **SUPPLIES** — between SUPPLIER and PROJECT
(which suppliers work with which projects)

Meaning

This model shows **individual pairwise relationships**, e.g.:

- Supplier X can supply Part Y
- Project Z uses Part Y

- Supplier X supplies Project Z

BUT — it **cannot specify** *which supplier provides which part to which project*.

Example (to show the issue)

Let's say:

- Supplier S1 can supply Part P1 and P2
- Project J1 uses P1
- Supplier S1 supplies to Project J1

Now — can we tell **whether S1 supplies P1 or P2 to J1?** ❌

No, because the relationships are separate; there's no place to tie them all together.

Limitation:

Binary relationships alone can't capture **three-way dependencies**.

That's why the ternary version (a) is more accurate for this scenario.

Figure 3.17(c): SUPPLY as a Weak Entity

This is an **alternative representation** of the ternary relationship using a **weak entity type**.

Entities:

1. SUPPLIER

- Attribute: Sname

2. PROJECT

- Attribute: Proj_name

3. PART

- Attribute: Part_no

4. SUPPLY — weak entity

- Attribute: Quantity

Relationships:

- **SS** between **SUPPLIER** and **SUPPLY**
- **SPJ** between **SUPPLY** and **PROJECT**
- **SP** between **SUPPLY** and **PART**

Meaning:

Here, **SUPPLY** is treated as an entity that *depends* on three owner entities:

- Supplier,
- Part,
- Project.

Each SUPPLY entity **exists only in the context of those three** (i.e., it cannot exist independently).

Cardinalities:

- SUPPLIER → SUPPLY : (1,N) → a supplier can appear in many supply records.
- PROJECT → SUPPLY : (1,N)
- PART → SUPPLY : (1,N)
- But for each SUPPLY instance, there is **exactly one supplier, one project, and one part**.

Key Concept:

A **weak entity** is an entity that does not have its own primary key and depends on "owner" entities.

Here, **SUPPLY** is uniquely identified by the combination:

(Sname, Part_no, Proj_name)

So this weak entity representation is **equivalent to the ternary relationship**, but explicitly models SUPPLY as an entity instead of a relationship.

COMPARISON SUMMARY

Feature	Figure (a)	Figure (b)	Figure (c)
Relationship type	Ternary	Binary (3 separate)	Weak Entity representation
Captures all 3-way dependency?	✓ Yes	✗ No	✓ Yes
Represents Quantity attribute?	✓ As relationship attribute	Not naturally	✓ As weak entity attribute
Meaning loss if decomposed?	✗ No	✓ Yes	✗ No
Implementation Equivalent to	SUPPLY(Sname, Part_no, Proj_name, Quantity)	3 tables, less meaningful	Weak entity dependent on 3

Concept Recap

Binary Relationship

- Involves **two entities**.
- Example: EMPLOYEE — DEPARTMENT (works_for)
- Easy to represent with a foreign key.

Ternary Relationship

- Involves **three entities simultaneously**.
- Example: SUPPLIER — PART — PROJECT (supplies)
- Needed when the relationship meaning depends on all three.

Weak Entity Representation

- Used when the relationship has **attributes** (like Quantity) or needs to be **uniquely identified** by multiple entities.
- Turns the ternary relationship into a dependent entity.

Real-World Analogy

Imagine a construction company:

- Suppliers provide **parts** (materials like cement, steel).
- Projects **use** parts.
- But which supplier provides **which part** to **which project** in **what quantity** — that's a *three-way dependency*.

If you only know pairwise links (supplier–project, project–part, supplier–part), you **can't tell** who supplied what to whom.

That's why we model **SUPPLY** as:

- A **ternary relationship**, or
- A **weak entity** depending on all three.

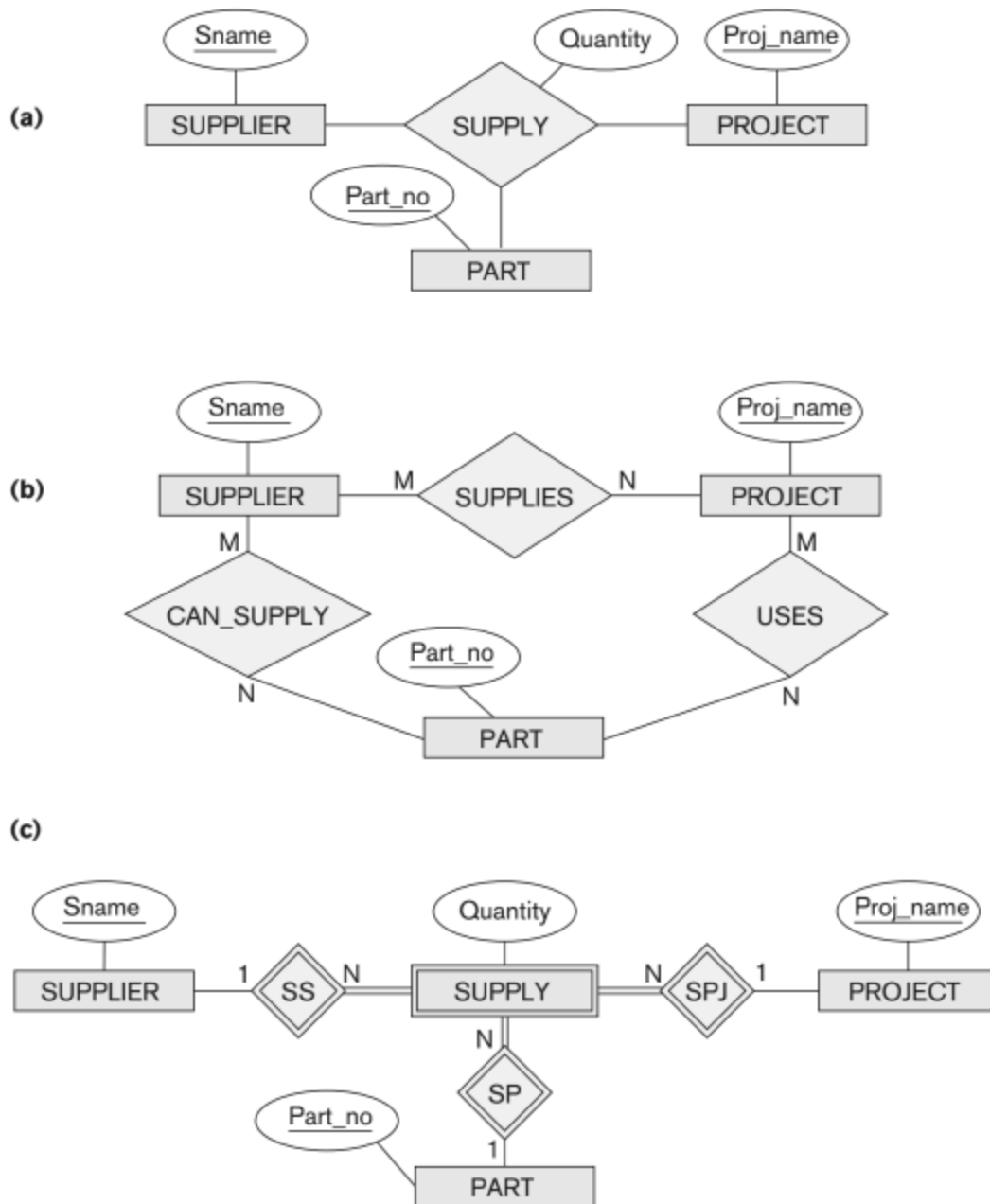
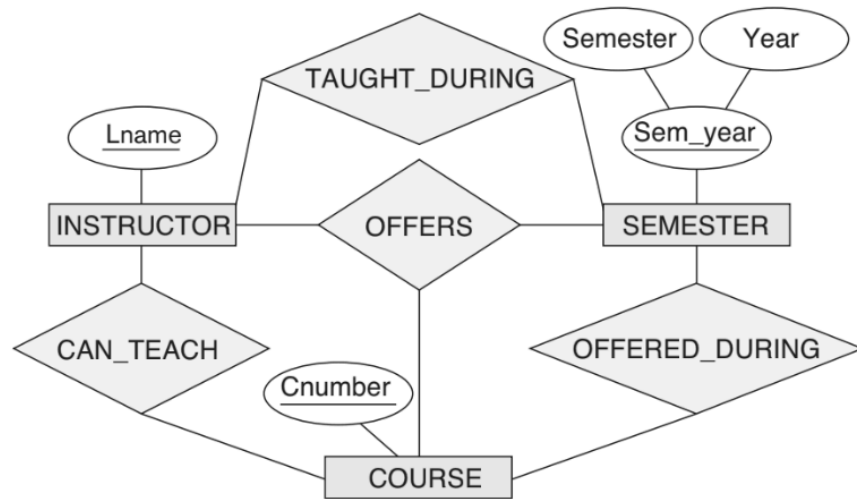


Figure 3.17

Ternary relationship types. (a) The SUPPLY relationship. (b) Three binary relationships not equivalent to SUPPLY. (c) SUPPLY represented as a weak entity type.

Example:

Figure 3.18
Another example of
ternary versus binary
relationship types.



Example:

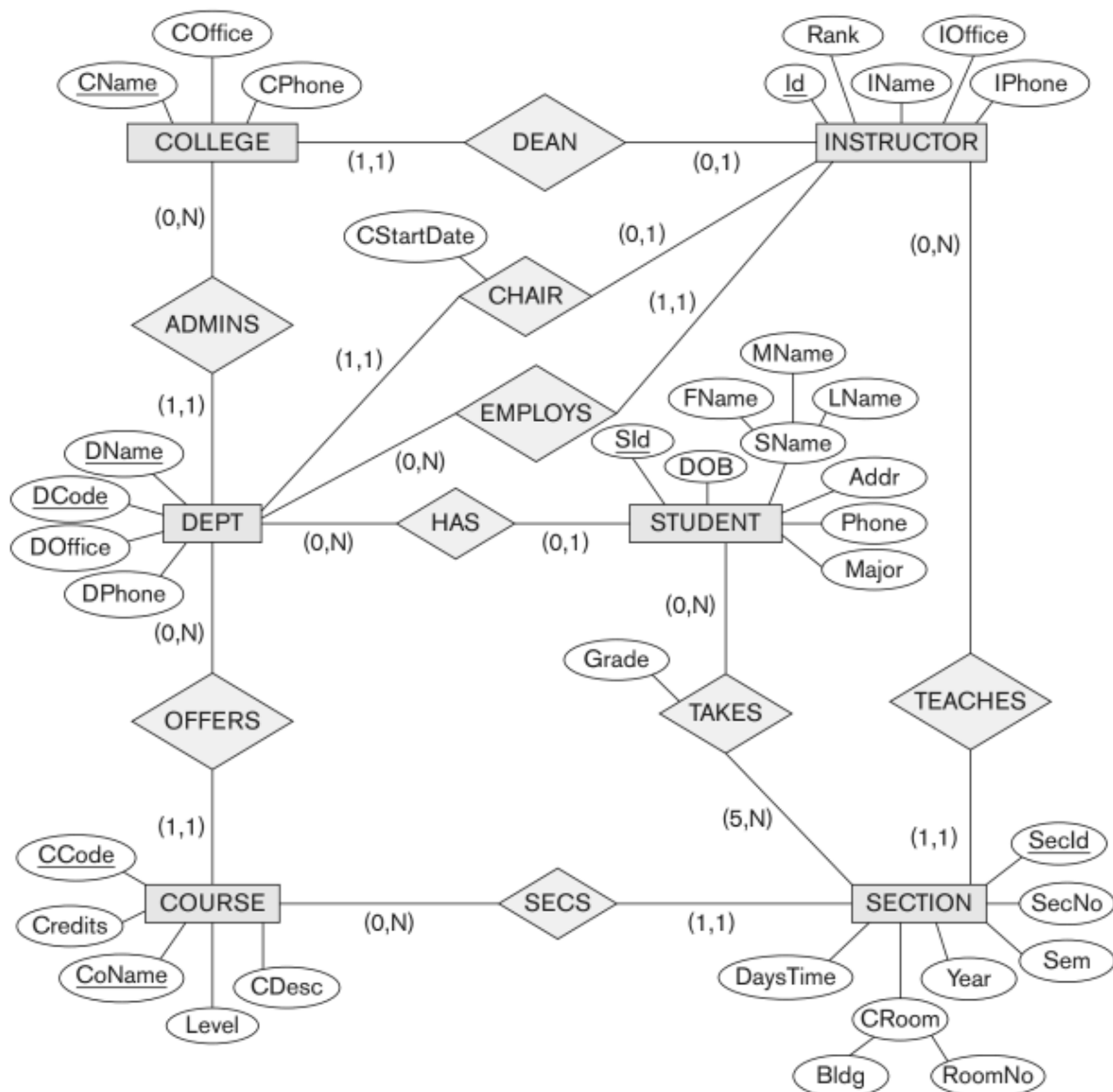


Figure 3.20
An ER diagram for a UNIVERSITY database schema.

This ERD models how a **University** manages data about:

- **Colleges**
- **Departments**
- **Courses**
- **Instructors**
- **Students**

- **Sections (classes)**
- And how they all relate (who teaches, who takes, etc.)

ENTITIES AND THEIR ATTRIBUTES

1. COLLEGE

Attributes:

- **CName** (college name)
- **COffice** (college office)
- **CPhone** (college phone)

Relationships:

- **Has one DEAN** (**DEAN**) $\rightarrow (1,1)$
- **Has many DEPARTMENTS** (**DEPT**) $\rightarrow (0,N)$

2. DEAN

Attributes:

- (None shown directly, but represents the head of a college)
- Relationship attribute: **CStartDate** (start date as dean)

Relationships:

- Each **DEAN** manages exactly **one COLLEGE** (1,1)
- Each **COLLEGE** has exactly **one DEAN** (1,1)

 **Relationship type:** One-to-One (1:1)

 **Relationship name:** **DEAN**

3. DEPARTMENT (DEPT)

Attributes:

- **DName** (department name)
- **DCode** (department code)

- **DOffice** (office)
- **DPhone** (phone number)

Relationships:

- **Belongs to one COLLEGE** (through **ADMINS**)
- **Employs INSTRUCTORS** (via **EMPLOYS**)
- **Has STUDENTS** (via **HAS**)
- **Offers COURSES** (via **OFFERS**)
- **Has one CHAIR** (via **CHAIR**)

4. CHAIR

Represents the relationship where one instructor serves as the **chairperson of a department**.

Relationships:

- Each **DEPARTMENT** has one **CHAIR (INSTRUCTOR)** (1,1)
- Each **INSTRUCTOR** may chair **zero or one DEPARTMENT** (0,1)

Attribute: **CStartDate** (chair start date)

 **Type:** One-to-One (1:1)

5. INSTRUCTOR

Attributes:

- **Id** (instructor ID)
- **IName** (name)
- **Rank**
- **IOffice**
- **IPhone**

Relationships:

- **EMPLOYED BY DEPARTMENT** (via **EMPLOYS**)

→ One DEPT employs many instructors (0,N)

- **TEACHES SECTIONS** (via **TEACHES**)
- **Can CHAIR one DEPARTMENT** (0,1)

6. EMPLOYS

A relationship between **DEPARTMENT** and **INSTRUCTOR**:

- Each **DEPARTMENT** employs many **INSTRUCTORS** (0,N)
- Each **INSTRUCTOR** works in exactly one **DEPARTMENT** (1,1)

 **Type:** One-to-Many (1:N)

7. STUDENT

Attributes:

- **SId** (student ID)
- **FName** , **MName** , **LName** (names)
- **SName** (student name or short form)
- **DOB**
- **Addr**
- **Phone**
- **Major**

Relationships:

- **Belongs to DEPARTMENT** (via **HAS**) → each student has one dept (0,1)
- **Takes SECTIONS** (via **TAKES**) → many courses (0,N)

8. HAS

Relationship between **DEPARTMENT** and **STUDENT**:

- Each **DEPARTMENT** has many **STUDENTS** (0,N)
- Each **STUDENT** belongs to one **DEPARTMENT** (0,1)

 **Type:** One-to-Many (1:N)

9. COURSE

Attributes:

- **CCode** (course code)
- **CoName** (course name)
- **Credits**
- **Level**
- **CDesc** (description)

Relationships:

- **Offered by DEPARTMENT** (via **OFFERS**)
- **Has many SECTIONS** (via **SECS**)

10. OFFERS

Relationship between **DEPARTMENT** and **COURSE**:

- Each **DEPARTMENT** offers many **COURSES** (0,N)
- Each **COURSE** is offered by one **DEPARTMENT** (1,1)

 **Type:** One-to-Many (1:N)

11. SECTION (SECS)

Represents **individual course sections** (specific class offerings).

Attributes:

- **SecId**
- **SecNo**
- **Sem** (semester)
- **Year**
- **CRoom** (classroom)
- **Bldg**
- **RoomNo**

- **DaysTime** (days and time)

Relationships:

- **Belongs to one COURSE** (1,1)
- **Taught by one INSTRUCTOR** (1,1) through **TEACHES**
- **Taken by many STUDENTS** (5,N) through **TAKES**

12. SECS

Relationship between **COURSE** and **SECTION**:

- Each **COURSE** has many **SECTIONS** (0,N)
- Each **SECTION** is for one **COURSE** (1,1)

 **Type:** One-to-Many (1:N)

13. TAKES

Relationship between **STUDENT** and **SECTION**:

- Each **STUDENT** takes many **SECTIONS** (0,N)
- Each **SECTION** has many **STUDENTS** (5,N)
- Attribute: **Grade**

 **Type:** Many-to-Many (M:N)

 **Attribute:** **Grade**

14. TEACHES

Relationship between **INSTRUCTOR** and **SECTION**:

- Each **INSTRUCTOR** teaches many **SECTIONS** (0,N)
- Each **SECTION** is taught by one **INSTRUCTOR** (1,1)

 **Type:** One-to-Many (1:N)

SUMMARY OF RELATIONSHIPS

Relationship	Between	Type	Meaning
DEAN	COLLEGE-DEAN	1:1	Each college has one dean
ADMINS	COLLEGE-DEPARTMENT	1:N	A college has many departments
CHAIR	DEPARTMENT-INSTRUCTOR	1:1	One instructor chairs a department
EMPLOYS	DEPARTMENT-INSTRUCTOR	1:N	A department employs many instructors
HAS	DEPARTMENT-STUDENT	1:N	A department has many students
OFFERS	DEPARTMENT-COURSE	1:N	A department offers many courses
SECS	COURSE-SECTION	1:N	A course has many sections
TEACHES	INSTRUCTOR-SECTION	1:N	An instructor teaches many sections
TAKES	STUDENT-SECTION	M:N	Students enroll in many sections
GRADE (attribute)	TAKES	–	Represents the student's performance

Conceptual Understanding

This ER diagram organizes **all academic data** into interconnected entities:

- **Administrative hierarchy:** College → Department → Course → Section
- **Academic roles:** Instructor (teaches), Student (takes)
- **Management:** Dean (college), Chair (department)
- **Relationships define cardinalities and participation constraints.**

Example:

Entities & main attributes

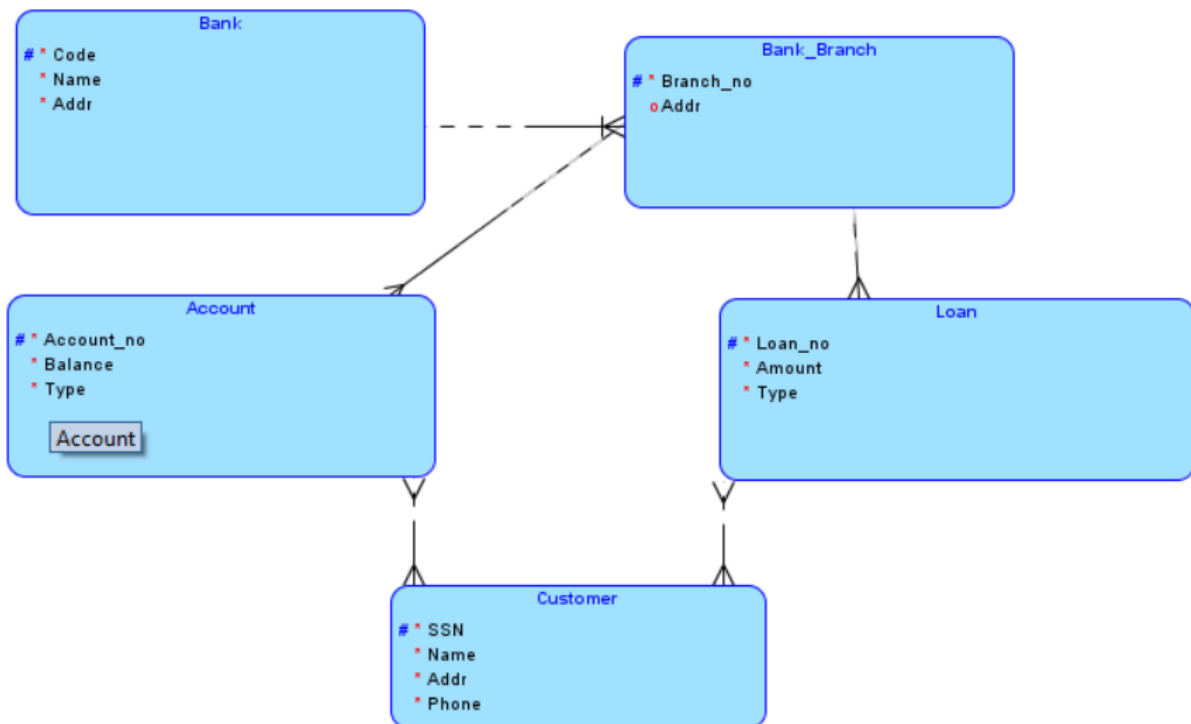
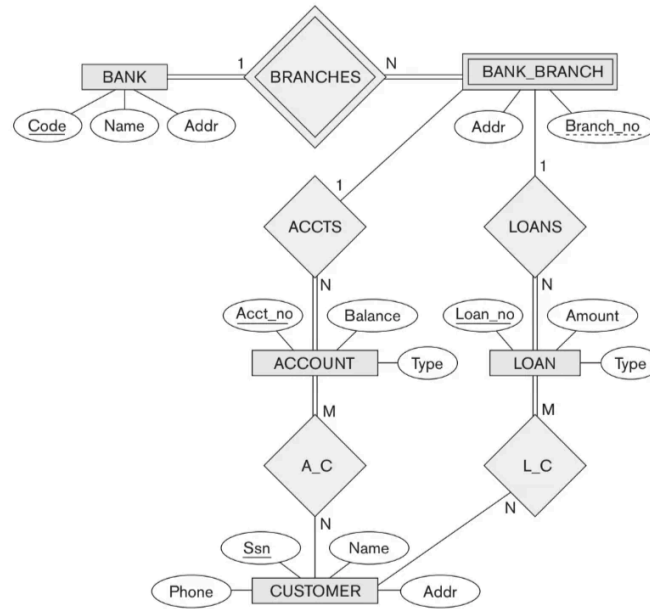
- **BANK** — attributes: `Code` (bank identifier / PK), `Name`, `Addr`.

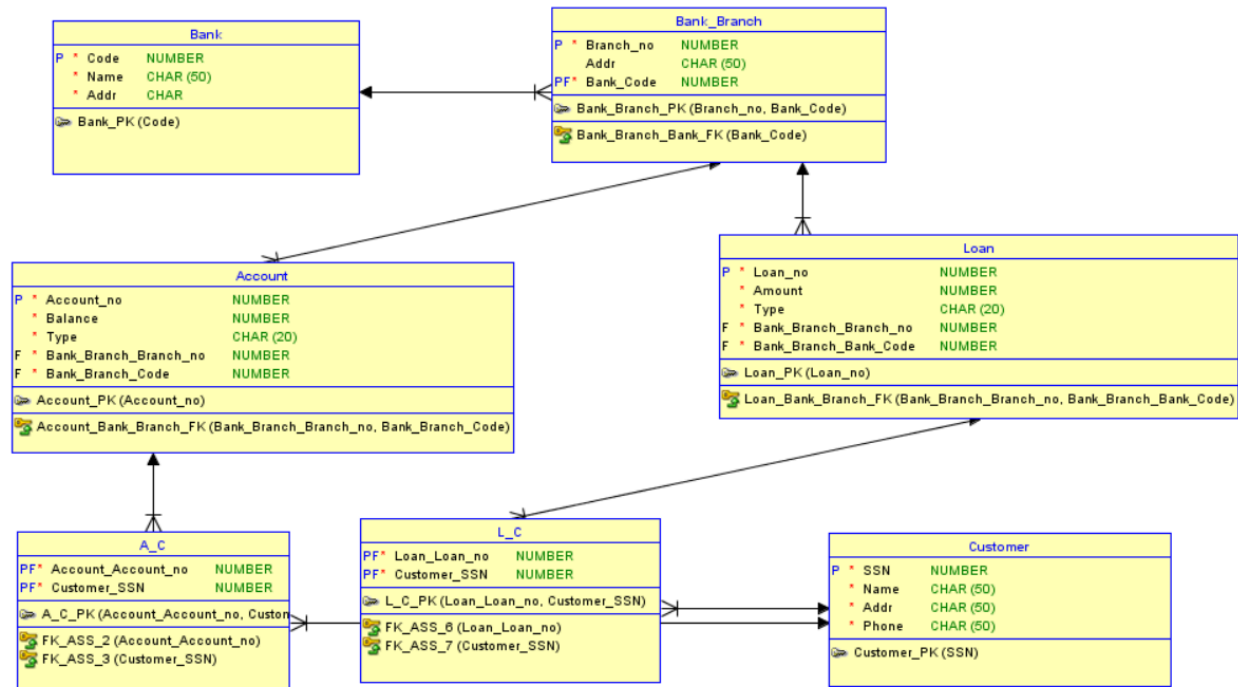
- **BANK_BRANCH** — *weak entity* (double rectangle). It depends on **BANK**. It has a **partial key** **Branch_no** (dashed underline in the diagram) and attribute **Addr**. **It is identified together with the BANK** (so the branch's full key = (Bank.Code, Branch_no)).
- **ACCOUNT** — attributes: **Acct_no** (PK), **Balance**, **Type**.
- **LOAN** — attributes: **Loan_no** (PK), **Amount**, **Type**.
- **CUSTOMER** — attributes: **Ssn** (PK), **Name**, **Phone**, **Addr**.

Relationships and cardinalities

- **BANK — BRANCHES — BANK_BRANCH** : 1 (BANK) : N (BANK_BRANCH). This is an *identifying* (owner → weak entity) relationship — parent's PK migrates into child PK.
- **BANK_BRANCH — ACCTS — ACCOUNT** : 1 (BANK_BRANCH) : N (ACCOUNT). Each branch has many accounts; each account belongs to one branch (non-identifying: account has its own PK **Acct_no**).
- **BANK_BRANCH — LOANS — LOAN** : 1 : N. Each branch issues many loans; loan belongs to one branch.
- **ACCOUNT — A_C — CUSTOMER** : M : N (many-to-many). A customer can have many accounts (joint accounts), and an account can have many customers. This will become a join/association table in the relational model.
- **LOAN — L_C — CUSTOMER** : M : N. Similar many-to-many (co-borrowers), will become a join table.

An ER diagram for a BANK database schema.





DDL GENERATED:

```

CREATE TABLE A_C
(
    Account_Account_no NUMBER NOT NULL ,
    Customer_SSN      NUMBER NOT NULL
);
ALTER TABLE A_C ADD CONSTRAINT A_C_PK PRIMARY KEY ( Account_Account_no, Customer_SSN );
  
```

```

CREATE TABLE Account
(
    Account_no          NUMBER NOT NULL ,
    Balance             NUMBER NOT NULL ,
    Type                CHAR (20) NOT NULL ,
    Bank_Branch_Branch_no NUMBER NOT NULL ,
    Bank_Branch_Code    NUMBER NOT NULL
);
ALTER TABLE Account ADD CONSTRAINT Account_PK PRIMARY KEY ( Account_no );
  
```

```
nt_no ) ;
```

```
CREATE TABLE Bank
```

```
(  
    Code NUMBER NOT NULL ,  
    Name CHAR (50) NOT NULL ,  
    Addr CHAR  
    -- WARNING: CHAR size not specified  
    NOT NULL  
);
```

```
ALTER TABLE Bank ADD CONSTRAINT Bank_PK PRIMARY KEY ( Code ) ;
```

```
CREATE TABLE Bank_Branch
```

```
(  
    Branch_no NUMBER NOT NULL ,  
    Addr CHAR (50) ,  
    Bank_Code NUMBER NOT NULL  
);
```

```
ALTER TABLE Bank_Branch ADD CONSTRAINT Bank_Branch_PK PRIMARY KEY ( Branch_no, Bank_Code ) ;
```

```
CREATE TABLE Customer
```

```
(  
    SSN NUMBER NOT NULL ,  
    Name CHAR (50) NOT NULL ,  
    Addr CHAR (50) NOT NULL ,  
    Phone CHAR (50) NOT NULL  
);
```

```
ALTER TABLE Customer ADD CONSTRAINT Customer_PK PRIMARY KEY ( SSN ) ;
```

```
CREATE TABLE L_C
```

```
(
    Loan_Loan_no NUMBER NOT NULL ,
    Customer_SSN NUMBER NOT NULL
);
ALTER TABLE L_C ADD CONSTRAINT L_C_PK PRIMARY KEY ( Loan_Loan_no,
Customer_SSN );
```

```
CREATE TABLE Loan
```

```
(
    Loan_no          NUMBER NOT NULL ,
    Amount           NUMBER NOT NULL ,
    Type             CHAR (20) NOT NULL ,
    Bank_Branch_Branch_no NUMBER NOT NULL ,
    Bank_Branch_Bank_Code NUMBER NOT NULL
);
ALTER TABLE Loan ADD CONSTRAINT Loan_PK PRIMARY KEY ( Loan_no );
```

```
ALTER TABLE Account ADD CONSTRAINT Account_Bank_Branch_FK FOREIGN
KEY ( Bank_Branch_Branch_no, Bank_Branch_Code ) REFERENCES Bank_Br
anch ( Branch_no, Bank_Code );
```

```
ALTER TABLE Bank_Branch ADD CONSTRAINT Bank_Branch_Bank_FK FOREIGN
KEY ( Bank_Code ) REFERENCES Bank ( Code );
```

```
ALTER TABLE A_C ADD CONSTRAINT FK_ASS_2 FOREIGN KEY ( Account_Acc
ount_no ) REFERENCES Account ( Account_no );
```

```
ALTER TABLE A_C ADD CONSTRAINT FK_ASS_3 FOREIGN KEY ( Customer_SS
N ) REFERENCES Customer ( SSN );
```

```
ALTER TABLE L_C ADD CONSTRAINT FK_ASS_6 FOREIGN KEY ( Loan_Loan_n
o ) REFERENCES Loan ( Loan_no );
```

```
ALTER TABLE L_C ADD CONSTRAINT FK_ASS_7 FOREIGN KEY ( Customer_SS
```

N) REFERENCES Customer (SSN) ;

ALTER TABLE Loan ADD CONSTRAINT Loan_Bank_Branch_FK FOREIGN KEY (Bank_Branch_Branch_no, Bank_Branch_Bank_Code) REFERENCES Bank_Branch (Branch_no, Bank_Code) ;

Concept	Description
No. of Tables	7
Weak Entity	BANK_BRANCH (identified by BANK)
Associative Entities	A_C , L_C
1-N Relationships	Bank → Branch, Branch → Account, Branch → Loan
M-N Relationships	Account ↔ Customer, Loan ↔ Customer
Generated Script	Correct and consistent with ER diagram

Example:

Your ER diagram consists of **three entities**:

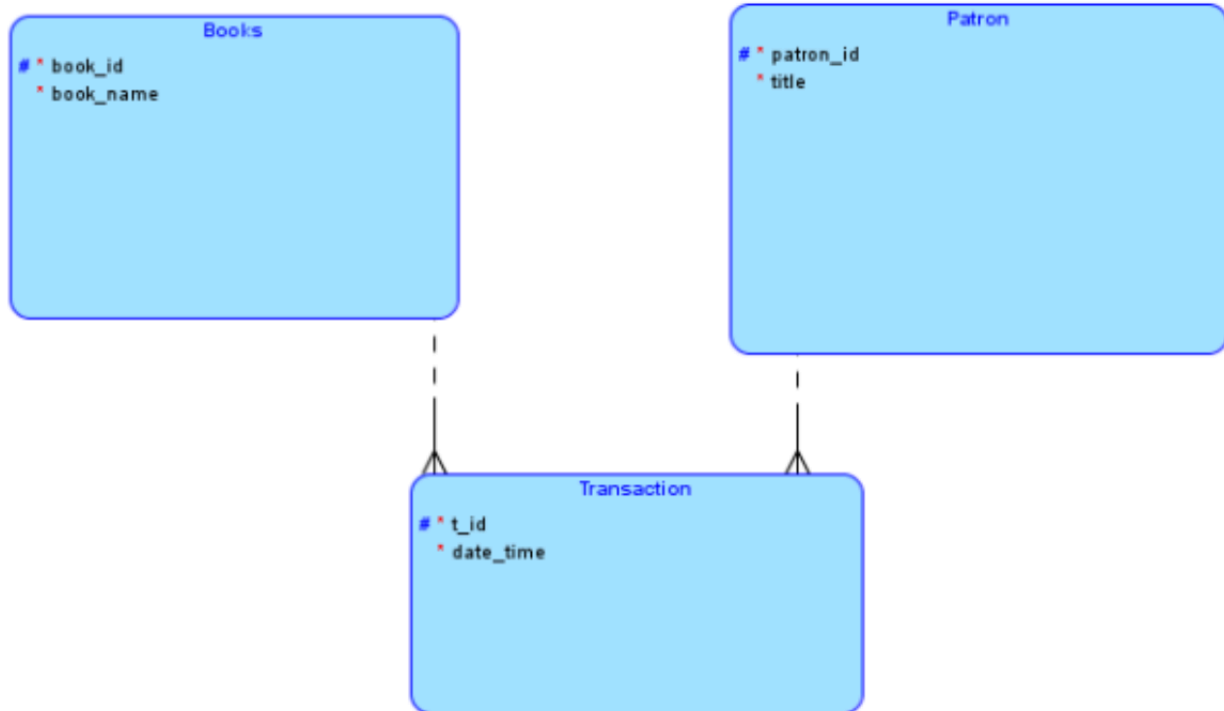
Entity	Key Attribute	Description
Books	book_id	Each record represents one book.
Patron	patron_id	Each record represents one library patron (user).
Transaction	t_id	Each record represents a book issue/return event between a Patron and a Book.

The diagram shows **two relationships**:

- **Books** → **Transaction** (one-to-many)
- **Patron** → **Transaction** (one-to-many)

That means:

- One **book** can be part of **many transactions** (issued multiple times).
- One **patron** can perform **many transactions** (borrow multiple books).





```

CREATE TABLE books (
book_id NUMBER NOT NULL,
book_name VARCHAR2(50) NOT NULL
);
  
```

```

ALTER TABLE books ADD CONSTRAINT books_pk PRIMARY KEY ( book_id );
  
```

```

CREATE TABLE patron (
patron_id NUMBER NOT NULL,
title VARCHAR2(50) NOT NULL
);
  
```

```

ALTER TABLE patron ADD CONSTRAINT patron_pk PRIMARY KEY ( patron_id
  
```

```
);
```

```
CREATE TABLE transaction (  
  t_id NUMBER NOT NULL,  
  date_time DATE NOT NULL,  
  books_book_id NUMBER NOT NULL,  
  patron_patron_id NUMBER NOT NULL  
);
```

```
ALTER TABLE transaction ADD CONSTRAINT transaction_pk PRIMARY KEY ( t  
_id );
```

```
ALTER TABLE transaction  
ADD CONSTRAINT transaction_books_fk FOREIGN KEY ( books_book_id )  
REFERENCES books ( book_id );
```

```
ALTER TABLE transaction  
ADD CONSTRAINT transaction_patron_fk FOREIGN KEY ( patron_patron_id )  
REFERENCES patron ( patron_id );
```

Example:

This diagram models a **university or college enrollment system** where:

- **Students** enroll in **Courses**.
- Each **Course** can have multiple **Sections** (like different times or instructors for the same course).
- Each **Student** can enroll in many **Course Sections**.
- The **final grade** a student earns is stored in the **Student_Course_Section** table.

So, this is a **many-to-many** relationship between **Students** and **Course Sections** - resolved through a **bridge table**.

ENTITY: STUDENT

- **Attributes:**

- `Student_Id` (Primary Key)
 - `First_Name`
 - `Last_Name`
 - `Phone`
- Meaning: Represents a student enrolled in the school.

ENTITY: COURSE

- **Attributes:**

- `Course_Id` (Primary Key)
 - `Course_Name`
 - `Description`
- Meaning: Each course offered by the institution, such as "Database Systems" or "Data Structures".

ENTITY: STUDENT

- **Attributes:**

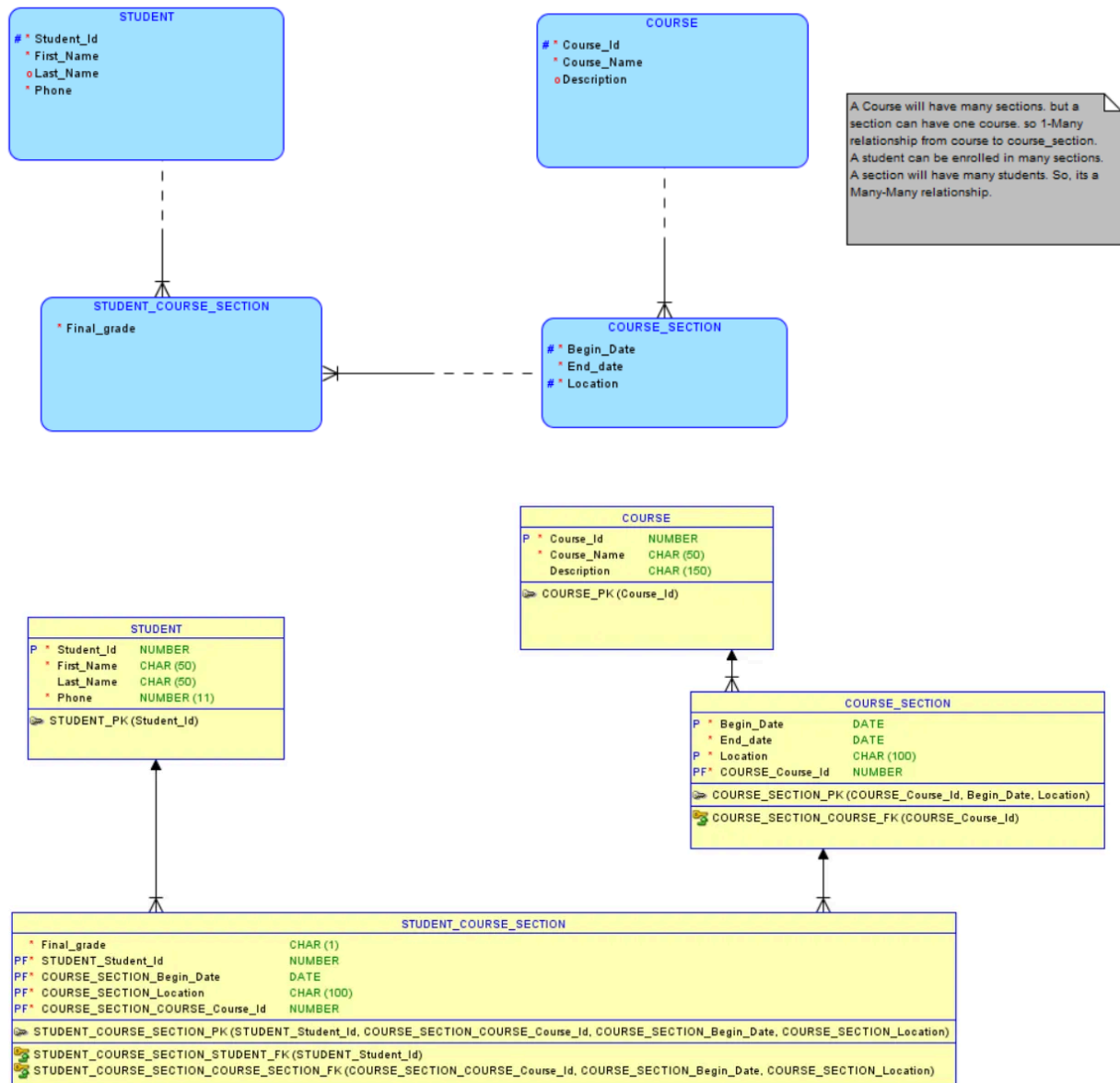
- `Student_Id` (Primary Key)
 - `First_Name`
 - `Last_Name`
 - `Phone`
- Meaning: Represents a student enrolled in the school.

RELATIONSHIP EXPLANATION

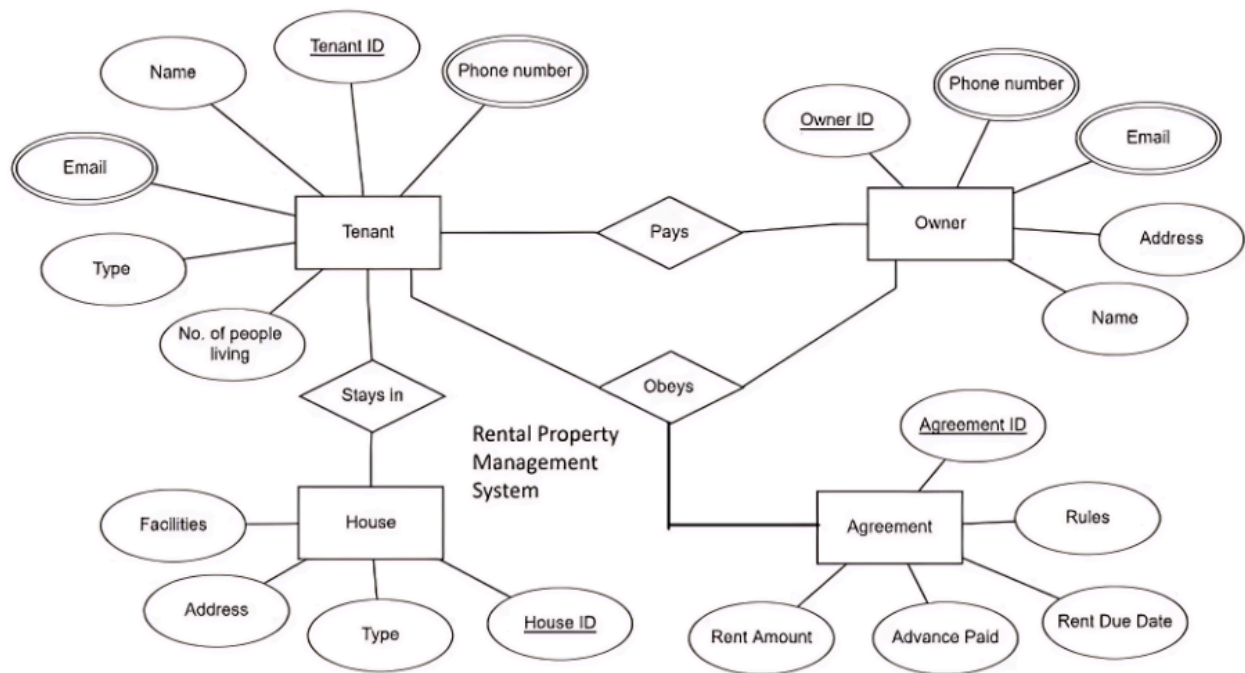
"A course will have many sections, but a section can have one course. So 1-many relationship from course to course_section. A student can be enrolled in many sections. A section will have many students. So, it is a many-many relationship."

This perfectly summarizes the logic:

- **Course ↔ Course_Section → 1 : M**
- **Student ↔ Course_Section → M : M** (resolved via Student_Course_Section)



Example:



Pays

- **Between:** Tenant and Agreement
- **Cardinality:** 1-M
- **Explanation:**
One **Tenant** can be involved in **multiple Agreements** (e.g., renting different houses over time), but each **Agreement** is paid by **one Tenant**.
So:
 - **Tenant (1) → Agreement (Many)**

Stays in

- **Between:** Tenant and House
- **Cardinality:** M-N (Many-to-Many)
- **Explanation:**
A **Tenant** can stay in multiple **Houses** over time (e.g., moving), and a **House** can host multiple **Tenants** (e.g., roommates).
So:

- **Tenant (Many) ↔ House (Many)**

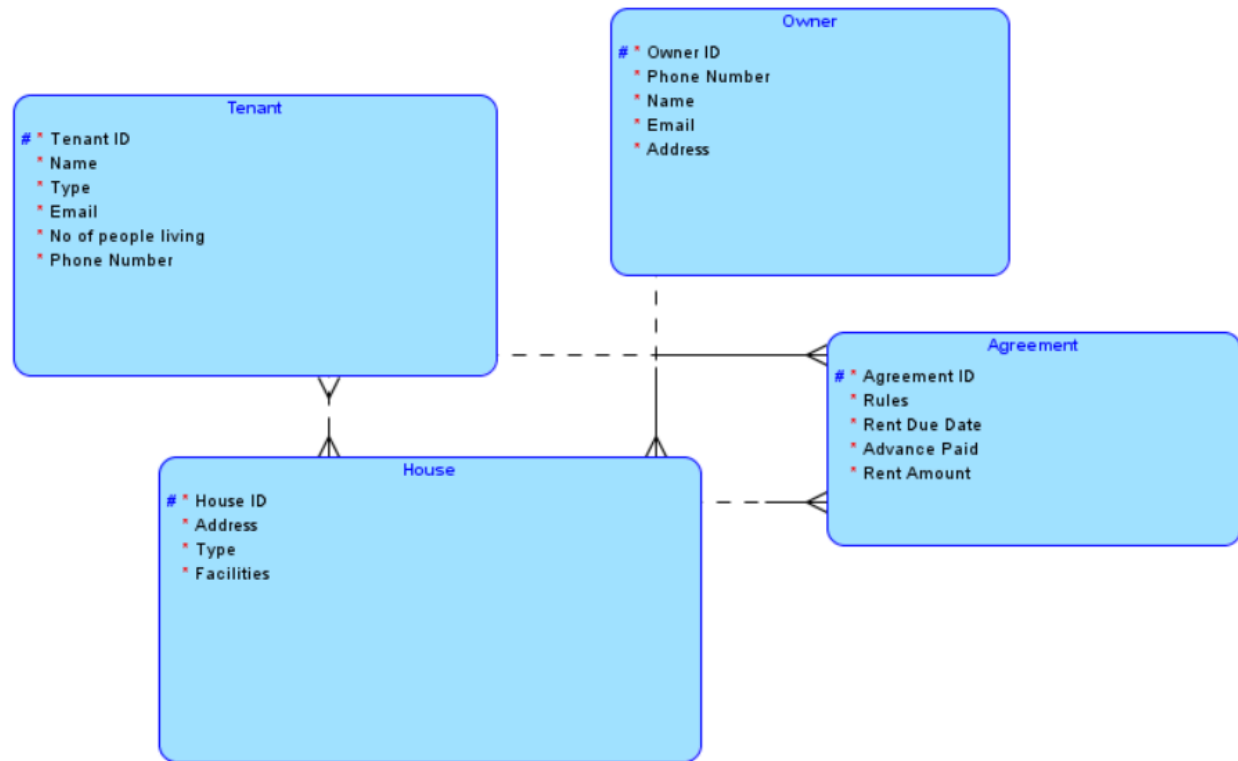
This requires a **junction table** like `Tenant_House` .

Owns

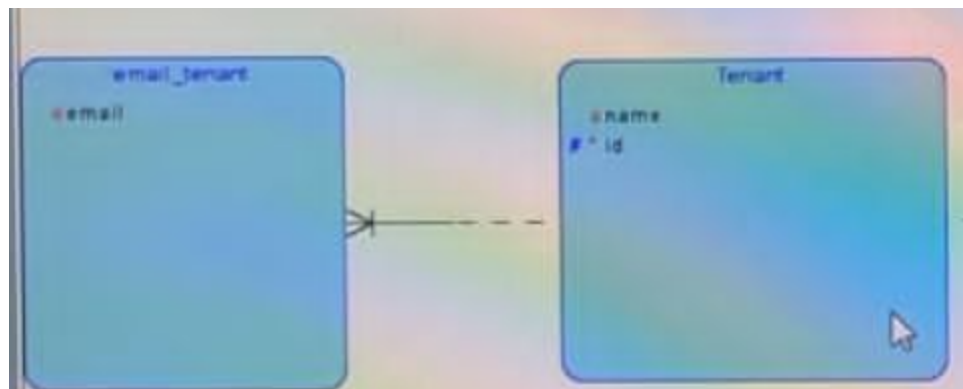
- **Between:** Owner and House
- **Cardinality:** 1-M
- **Explanation:**
One **Owner** can own multiple **Houses**, but each **House** is owned by only **one Owner**.
So:
 - **Owner (1) → House (Many)**

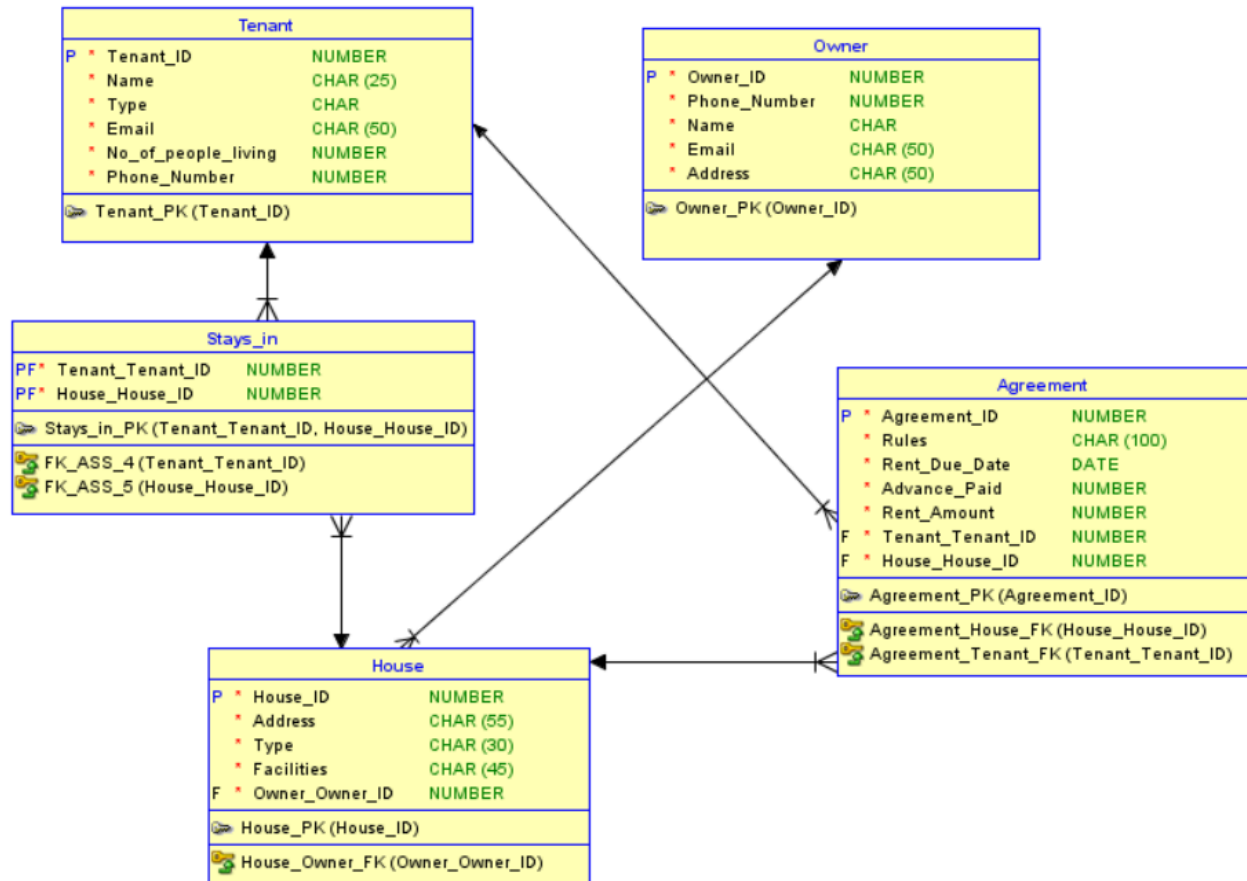
Agreement

- **Between:** House and Agreement
- **Cardinality:** 1-M
- **Explanation:**
One **House** can have multiple **Agreements** over time (e.g., different tenants), but each **Agreement** is tied to **one House**.
So:
 - **House (1) → Agreement (Many)**



For any multi-valued attribute make a new entity and show 1-MANY relation.





Generated DDL:

```
CREATE TABLE Agreement
(
  Agreement_ID    NUMBER NOT NULL ,
  Rules          CHAR (100) NOT NULL ,
  Rent_Due_Date  DATE NOT NULL ,
  Advance_Paid   NUMBER NOT NULL ,
  Rent_Amount    NUMBER NOT NULL ,
  Tenant_Tenant_ID NUMBER NOT NULL ,
  House_House_ID  NUMBER NOT NULL
);
ALTER TABLE Agreement ADD CONSTRAINT Agreement_PK PRIMARY KEY ( Agreement_ID );
```

```
CREATE TABLE House
(
    House_ID    NUMBER NOT NULL ,
    Address     CHAR (55) NOT NULL ,
    Type        CHAR (30) NOT NULL ,
    Facilities   CHAR (45) NOT NULL ,
    Owner_Owner_ID NUMBER NOT NULL
);
ALTER TABLE House ADD CONSTRAINT House_PK PRIMARY KEY ( House_ID
);
```

```
CREATE TABLE Owner
(
    Owner_ID    NUMBER NOT NULL ,
    Phone_Number NUMBER NOT NULL ,
    Name        CHAR
    -- WARNING: CHAR size not specified
    NOT NULL ,
    Email       CHAR (50) NOT NULL ,
    Address     CHAR (50) NOT NULL
);
ALTER TABLE Owner ADD CONSTRAINT Owner_PK PRIMARY KEY ( Owner_ID
);
```

```
CREATE TABLE Stays_in
(
    Tenant_Tenant_ID NUMBER NOT NULL ,
    House_House_ID   NUMBER NOT NULL
);
ALTER TABLE Stays_in ADD CONSTRAINT Stays_in_PK PRIMARY KEY ( Tenant
_Tenant_ID, House_House_ID );
```

```
CREATE TABLE Tenant
```

```
(
  Tenant_ID NUMBER NOT NULL ,
  Name    CHAR (25) NOT NULL ,
  Type    CHAR
  -- WARNING: CHAR size not specified
  NOT NULL ,
  Email    CHAR (50) NOT NULL ,
  No_of_people_living NUMBER NOT NULL ,
  Phone_Number    NUMBER NOT NULL
);
ALTER TABLE Tenant ADD CONSTRAINT Tenant_PK PRIMARY KEY ( Tenant_ID );
```

```
ALTER TABLE Agreement ADD CONSTRAINT Agreement_House_FK FOREIGN
KEY ( House_House_ID ) REFERENCES House ( House_ID );
```

```
ALTER TABLE Agreement ADD CONSTRAINT Agreement_Tenant_FK FOREIGN
KEY ( Tenant_Tenant_ID ) REFERENCES Tenant ( Tenant_ID );
```

```
ALTER TABLE Stays_in ADD CONSTRAINT FK_ASS_4 FOREIGN KEY ( Tenant_Tenant_ID ) REFERENCES Tenant ( Tenant_ID );
```

```
ALTER TABLE Stays_in ADD CONSTRAINT FK_ASS_5 FOREIGN KEY ( House_House_ID ) REFERENCES House ( House_ID );
```

```
ALTER TABLE House ADD CONSTRAINT House_Owner_FK FOREIGN KEY ( Owner_Owner_ID ) REFERENCES Owner ( Owner_ID );
```

3.28. Consider the ER schema for the MOVIES database in Figure 3.25. Assume that MOVIES is a populated database. ACTOR is used as a generic term and includes actresses. Given the constraints shown in the ER schema, respond to the following statements with True, False, or Maybe. Assign a response of Maybe to statements that, although not explicitly shown to be True, cannot be

proven False based on the schema as shown. Justify each answer.

- a. There are no actors in this database that have been in no movies.
- b. There are some actors who have acted in more than ten movies.
- c. Some actors have done a lead role in multiple movies.
- d. A movie can have only a maximum of two lead actors.
- e. Every director has been an actor in some movie.
- f. No producer has ever been an actor.
- g. A producer cannot be an actor in some other movie.
- h. There are movies with more than a dozen actors.
- i. Some producers have been a director as well.
- j. Most movies have one director and one producer.
- k. Some movies have one director but several producers.
- l. There are some actors who have done a lead role, directed a movie, and produced a movie.
- m. No movie has a director who also acted in that movie.

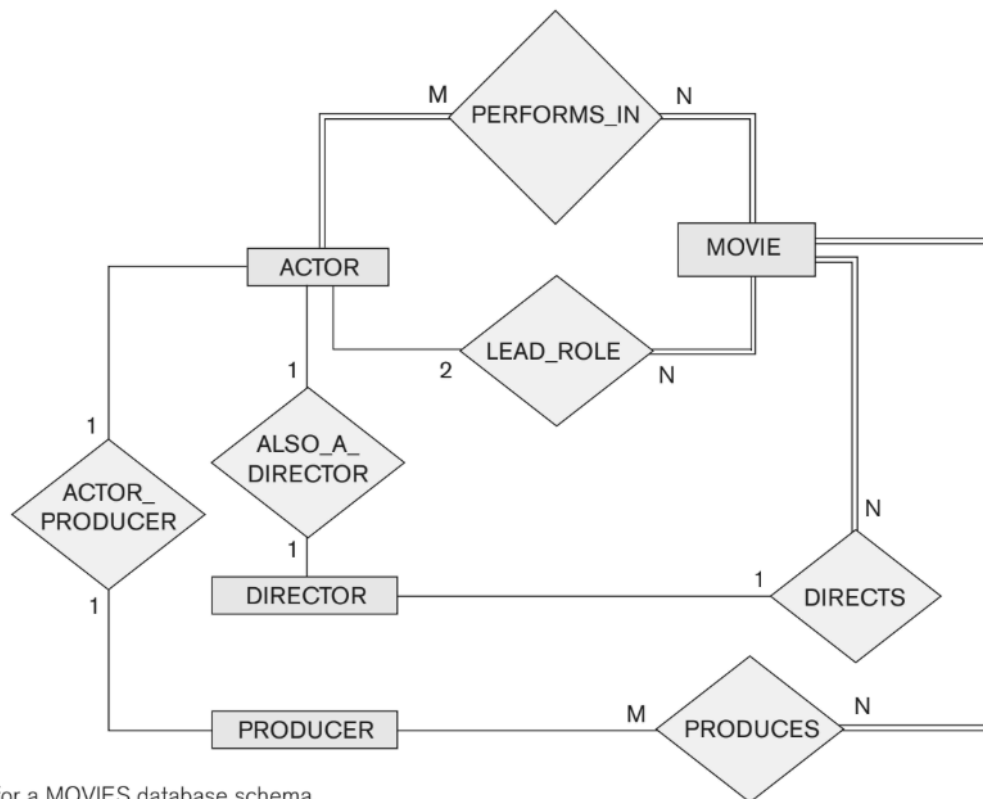


Figure 3.25
An ER diagram for a MOVIES database schema.

Statement	Answer	Justification (Short)
(a)	False	Actor participation in movies not mandatory
(b)	Maybe	Could act in >10 movies, not certain
(c)	True	Actor can have multiple lead roles
(d)	True	Max 2 lead actors per movie
(e)	False	Not all directors are actors
(f)	False	Actors can also be producers
(g)	False	Producers can act in other movies
(h)	Maybe	No limit on # of actors per movie
(i)	Maybe	Possible but not guaranteed
(j)	Maybe	One director yes, one producer not guaranteed
(k)	True	One director, many producers allowed
(l)	Maybe	Possible combo, not certain
(m)	False	No restriction preventing it

3.27. Cardinality ratios often dictate the detailed design of a database. The cardinality ratio depends on the real-world meaning of the entity types involved and is defined by the specific application. For the following binary relationships, suggest cardinality ratios based on the common-sense meaning of the entity types. Clearly state any assumptions you make.

Entity 1	Cardinality Ratio	Entity 2
1. STUDENT	_____	SOCIAL_SECURITY_CARD
2. STUDENT	_____	TEACHER
3. CLASSROOM	_____	WALL
4. COUNTRY	_____	CURRENT_PRESIDENT
5. COURSE	_____	TEXTBOOK
6. ITEM (that can be found in an order)	_____	ORDER
7. STUDENT	_____	CLASS
8. CLASS	_____	INSTRUCTOR
9. INSTRUCTOR	_____	OFFICE
10. EBAY_AUCTION_ITEM	_____	EBAY_BID

- **STUDENT — 0..1 : 1 — SOCIAL_SECURITY_CARD**

Assumption: a social-security card is issued to exactly one person; a student may or may not have an SSN (e.g., non-residents). So a card belongs to one student; a student has at most one card.

- **STUDENT — M : N — TEACHER**

Assumption: over time a student takes courses from many teachers and a teacher instructs many students → many-to-many.

- **CLASSROOM — 1 : N — WALL**

Assumption: a classroom typically has several walls (e.g., 4); each wall belongs to one classroom (walls are not shared between distinct classrooms).

- **COUNTRY — 0..1 : 1 — CURRENT_PRESIDENT**

Assumption: a country has at most one current president (some countries have none or a monarch); a given president is the current president of exactly one country.

- **COURSE — M : N — TEXTBOOK**

Assumption: a course may use zero, one, or several textbooks; a textbook may be used by many courses → many-to-many (optionally allowing 0 on either side).

- **ITEM (product) — M : N — ORDER**

Assumption: an item/product can appear in many orders; an order contains many items → many-to-many (if "item" meant "order line" this could be different, but we assume item = product).

- **STUDENT — M : N — CLASS**

Assumption: a student enrolls in many classes and each class has many students (standard enrollment model).

- **CLASS — N : 1 — INSTRUCTOR**

(Equivalently: **INSTRUCTOR 1 : N CLASS.**)

Assumption: each class/section is taught by one primary instructor (co-teaching is possible but not assumed); an instructor can teach many classes.

- **INSTRUCTOR — 0..1 : 0..1 — OFFICE**

Assumption: offices are typically one-to-one but optional — an instructor may have no assigned office (e.g., adjuncts) and some offices may be unassigned or shared only rarely. If you **assume no sharing and everyone has an office**, this becomes 1:1.

- **EBAY_AUCTION_ITEM — 1 : N — EBAY_BID**

Assumption: each auction item can receive many bids; each bid is placed on exactly one auction item.

Q: Cases Where Use of a NULL Value is Appropriate

A **NULL** in a database represents a **missing, unknown, inapplicable, or undefined value** — not zero or blank.

Appropriate Cases:

1. **Value Unknown at the Time of Data Entry:**

- Example: A student's phone number not yet provided.

2. Value Not Applicable:

- Example: "Spouse name" field for an unmarried person.

3. Value Exists but Not Recorded Yet:

- Example: Employee's performance review pending.

4. Attribute Optional in the Relationship:

- Example: Middle name of a person (some people don't have one).

5. Data Not Available from Source:

- Example: Supplier's fax number missing from the received data file.

6. To Indicate Absence of a Relationship:

- Example: Manager ID in an employee table is NULL for the CEO (no manager).

Q: Consider the ER diagram shown in Figure 3.22 for part of a BANK database. Each bank can have multiple branches, and each branch can have multiple accounts and loans.

a. List the strong (nonweak) entity types in the ER diagram.

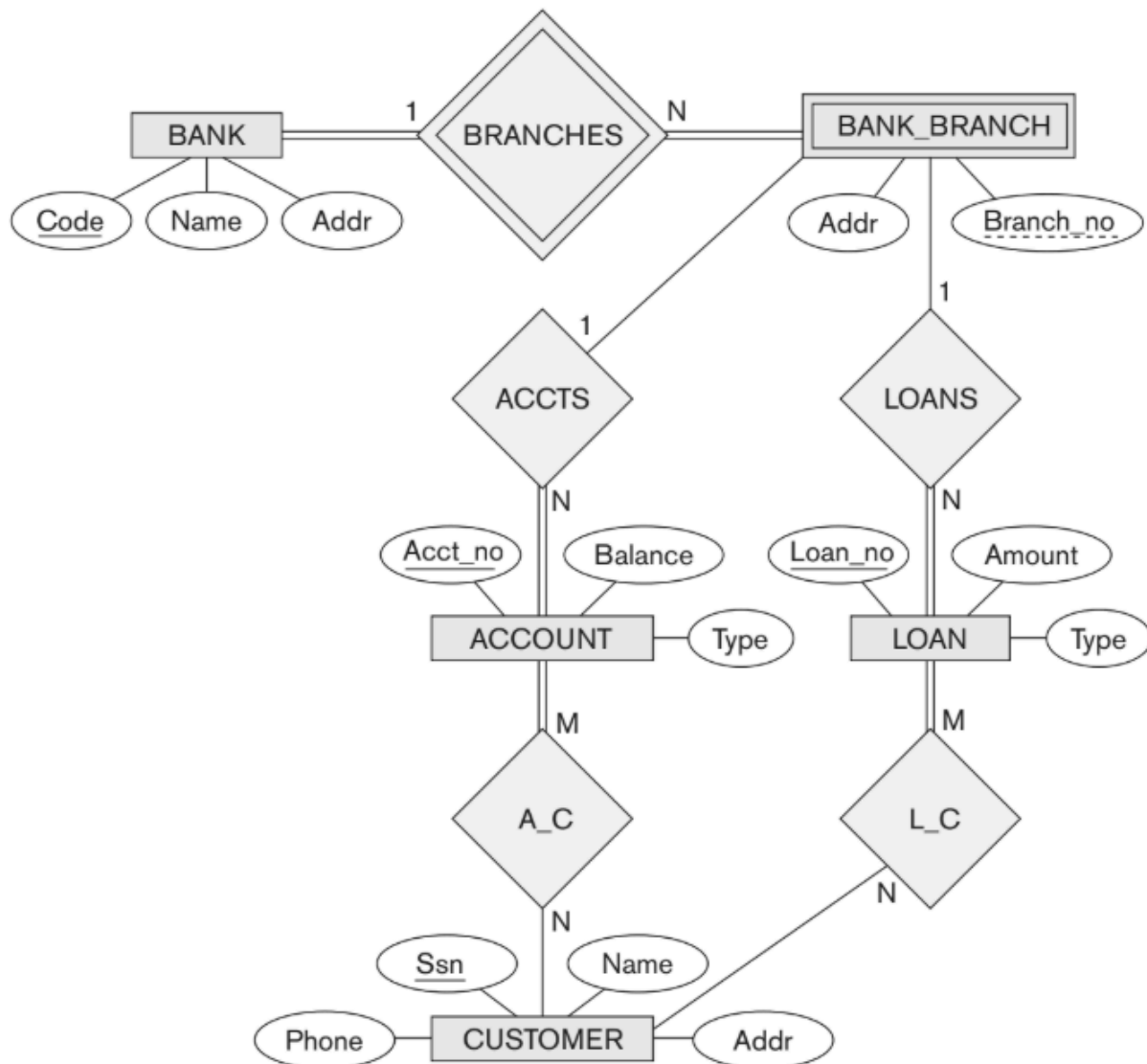
b. Is there a weak entity type? If so, give its name, partial key, and identifying relationship.

c. What constraints do the partial key and the identifying relationship of the weak entity type specify in this diagram?

d. List the names of all relationship types, and specify the (min, max) constraint on each participation of an entity type in a relationship type. Justify your choices.

Figure 3.22

An ER diagram for a BANK database schema.



- A **partial key** is an attribute (or set of attributes) of the weak entity that uniquely identifies weak-entities **only within the scope of a particular owner entity**.
- The **identifying relationship** binds the weak entity to its owner (or owners). The primary (global) key of the weak entity is formed by **concatenating the owner's key** with the weak entity's partial key.
- The identifying relationship is **mandatory (total) on the weak-entity side**: every weak-entity instance must be related to some owner instance (i.e., a

branch must be related to a bank; an account must be related to a branch; a loan must be related to a branch).